PIXELROOT: Hardware-Anchored, Sensor-Level Media Provenance with Real-Time Blockchain Notarization for Trustworthy Imaging

Mihir Kavishwar

***Abstract*—The realism of AI-generated "deepfake" imagery has outpaced society's ability to tell synthetic media from camera-captured reality. Detectors generalize poorly to unseen generators, and provenance schemes that bind trust to *metadata* fail once the metadata is stripped. We present PIXELROOT, which moves the root of trust into the *pixels* and onto a public blockchain. At capture, a CMOS sensor embeds a 128-bit, Reed–Solomon-coded provenance payload through a keyed, low-frequency per-region gain modulation. Each sensor holds a signing key generated on-die and certified on the blockchain by its manufacturer; it signs a capture record that binds the payload to the frame's hashes and to a robust content fingerprint, and the record is notarized on the same blockchain, which also records time-scoped revocation. We contribute a formal architecture and threat model with five security propositions; a measured robustness study on the Kodak images with real codecs, recovering the payload on 22–$23/24$ images down to JPEG $Q20$ ($21/24$ at $Q10$) at 38 dB PSNR with $0/1224$ false accepts; a device-identity evaluation showing that the in-pixel mark alone can be copied without any key (78% of trials) while the content-bound record rejects every cross-scene copy and large splice on held-out images ($0/180$) and lets 90% of processed genuine copies verify, linking local edits back to the original; a tested registry contract (47k gas per batch, $O(1)$ per image); a CMOS-manufacturing feasibility analysis of per-pixel gain; and a privacy-preserving notarization design with a phased adoption path toward a cross-OEM interoperability standard.**

***Index Terms*—media provenance, deepfakes, blockchain, CMOS image sensor, PRNU, digital watermarking, content authenticity, C2PA, image forensics, JPEG.**

I. INTRODUCTION

DIGITAL images and videos have become primary evidence in journalism, courts, insurance, humanitarian monitoring, and everyday social discourse. That evidentiary role is collapsing. Diffusion models, generative adversarial networks, and neural rendering now synthesize photorealistic faces, scenes, and full videos that humans—and automated detectors—routinely fail to distinguish from reality [1], [2]. The harm is no longer hypothetical. Industry incident trackers attribute on the order of $1.2–1.3 billion in documented fraud losses to deepfakes in 2025, rising steeply year over year, with the overwhelming majority of losses originating on social-media platforms [3], [4]; enterprise surveys report that a majority of organizations experienced deepfake-related incidents with mean losses exceeding $280,000 [5]. Beyond fraud, non-consensual intimate imagery and election disinformation erode personal safety and institutional trust.

The dominant technical response has been *reactive detection*: train a classifier to spot the statistical fingerprints of a given generator. This strategy is structurally disadvantaged. Detectors overfit to the artifacts of the generators in their training set and degrade sharply on unseen models—recent in-the-wild benchmarks report AUC drops of roughly 45–50% relative to academic datasets [1], [6]. Because every published detector also serves as a differentiable signal that the next generation of generators can be trained to defeat, detection is a perpetual "tug of war" that the defender is not guaranteed to win [7], [8].

An alternative philosophy inverts the problem: instead of trying to prove a piece of media is *fake*, prove that a piece of media is *real* by binding it, at creation time, to a trustworthy origin. This is the *provenance* paradigm now embodied by the C2PA Content Credentials standard and by secure-capture products such as Truepic and ProofMode [9]–[12]. These systems are a major advance—C2PA supports hardware-backed signing, rich edit-history assertions, and ingredient hashes, and our goal is to *complement* rather than replace it. But the authenticity claim is carried, in the common case, in a *signed metadata container* attached to the file, and that container is a single point of failure: metadata can be deliberately stripped by platforms, lost in transcoding, or, worse, the binding itself can be attacked. In September 2025, a vulnerability in the Nikon Z6 III's C2PA implementation let a camera sign fabricated composites as authentic, forcing the vendor to suspend its authenticity service and revoke every certificate it had issued [13]. If the credential is removed, the pixels carry no inherent evidence of their origin. PIXELROOT targets exactly this gap, and can populate C2PA assertions while adding an in-pixel, ledger-witnessed binding that survives container loss.

A. The PIXELROOT thesis

We argue that a robust root of trust for visual media should live where the information actually is: in the pixels, anchored to physical hardware, and witnessed by an immutable public ledger. PIXELROOT embeds a provenance payload *into the pixel values* at the moment of capture through a keyed, low-frequency *per-region* sensor gain modulation—a perturbation small enough to be imperceptible yet structured to survive lossy re-encoding—and *simultaneously* notarizes a

M. Kavishwar is an independent researcher with the PixelRoot Project (e-mail: hello@mihirkavishwar.com).

Manuscript prepared for submission, October 2026. The author declares no conflict of interest. This work received no external funding.

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

cryptographic commitment of the image to a blockchain. This yields two independent bindings:

- a **hard binding**—a SHA-256 commitment recorded on-chain, giving an immutable, publicly verifiable timestamp of existence; and
- a **soft, in-pixel binding**—a sparse, error-corrected signature tied to the sensor's factory identity that travels *with the pixels* even if the file container and its metadata are destroyed.

Both bindings are anchored to the **sensor's own identity**: a signing key generated inside the sensor at manufacture, never exported (not even to the manufacturer), and certified on the same blockchain. The sensor signs a capture record that ties the in-pixel payload to the frame's hashes and to a robust fingerprint of its content, so a payload copied onto other content no longer verifies (§V-G).

The two bindings are complementary: the hard binding gives cryptographic non-repudiation but is destroyed by any re-encoding; the in-pixel binding tolerates mild processing and survives metadata stripping, but is statistically softer. Together they cover each other's failure modes.

B. Contributions

1) **A unifying survey and gap analysis** (§III) spanning deepfake generation/detection, passive PRNU forensics, active watermarking, provenance standards, and blockchain media integrity, showing that no prior line of work simultaneously achieves (a) a hardware root of trust, (b) survival of metadata stripping, and (c) public, real-time notarization.
2) **A formal architecture with a feasibility-grounded hardware ask** (§V): a sensor-gain *feasibility spectrum* (global → per-column → per-region → per-pixel) that grounds the embedding in the low-frequency, per-column/per-region gain control that column-parallel sensors can realistically provide, plus the 128-bit payload, keyed spread-spectrum carrier selection, error-correction, and a smart-contract notarization protocol.
3) **A CMOS-manufacturing feasibility analysis of per-pixel gain** (§V-B): a survey-grounded argument that per-pixel intensity control is manufacturable—not a physics barrier—via floating-diffusion conversion-gain selection, coded-exposure pixels, and 3D-stacked per-pixel ADCs joined by Cu–Cu hybrid bonding, with *engineering schematics* (Figs. 3–5), an explicit cost/trade-off assessment and a concrete OEM "provenance-write" primitive; and the optional per-pixel tamper-localization layer this unlocks.
4) **A device-identity layer on the ledger** (§V-G): manufacturer-certified production lots of on-die sensor keys, time-scoped device and manufacturer revocation, chip-signed capture records bound to a content fingerprint, and a block anchor that brackets capture time from below—with a tested smart contract (§V-H) and an end-to-end evaluation showing that the in-pixel mark alone is copyable and that the content-bound record stops identity transplant (§IX-D).
5) **A threat model and security analysis** (§IV, §VI) with five propositions and a threat catalogue covering re-capture, metadata forgery, replay, compression-laundering, deepfake substitution, splicing, the watermark copy attack, sensor-key compromise, and the PRNU fingerprint-copy attack.
6) **A measured robustness study on real images** (§VII, §IX) of the in-pixel channel on the 24 Kodak photographs with *real libjpeg/WebP* codecs, a *real Reed–Solomon* code, and a WAVES-style attack suite (JPEG, WebP, noise, blur, resize, photometric, crop)—reporting real PSNR/SSIM, BER–quality curves, an amplitude operating curve, wrong-key/forgery/copy-attack false-accept rates, a real-splice tamper-localization measurement (ROC-AUC/F1/IoU), and a passive-PRNU baseline, together with an explicit account of the geometric limits.
7) **A deployable systems design** (§X): privacy-preserving notarization that keeps *no raw PII on-chain* (opaque record hashes, optional location, off-chain records that can be erased), a justification of public-ledger anchoring over a single RFC 3161 time-stamping authority (with a hybrid option), and a phased, backward-compatible adoption path toward an OEM standard.

II. Background

A. The CMOS imaging pipeline

A modern camera forms an image by integrating photo-generated charge in each sensor photosite, converting it to a voltage, applying analog gain, and digitizing it. Manufacturing imperfections make each photosite's response slightly non-uniform; this multiplicative *photo-response non-uniformity* (PRNU) is a stable, device-unique noise pattern that has long been used as a forensic "sensor fingerprint" [14], [15]. Crucially for PIXELROOT, modern column-parallel CMOS architectures already place a programmable-gain amplifier and ADC *per column*, support per-region exposure/gain in high-dynamic-range and dual-conversion-gain modes, and embed a factory-burned identifier in one-time-programmable (OTP) memory. PIXELROOT's robust channel does *not require* the per-pixel gain control that commercial sensors do not yet expose—though we show per-pixel control is manufacturable and unlocks an extra localization layer (§V-B); instead it repurposes the coarse, low-frequency gain controls that are realistic today (or a modest standardizable extension of them, §V-A): rather than only *reading* the passive PRNU after the fact, it *actively writes* a chosen, error-corrected, low-frequency signal into the image during exposure and ties that signal to the OTP identity.

B. Lossy compression: JPEG and MPEG/H.26x

Captured media is almost never stored or transmitted losslessly. JPEG applies a block DCT, quantizes coefficients with a quality-dependent matrix that preferentially discards high-frequency energy, and entropy-codes the result [16].

Video codecs (MPEG-2, H.264/AVC, HEVC) add motion-compensated inter-frame prediction, integer transforms, in-loop deblocking, and group-of-pictures (GOP) structure [17], [18]. All of these operations are *lossy* and spatially redistributive, which is exactly why any embedded signal must be designed to survive quantization—an issue we analyze in §VII.

C. Cryptographic and blockchain primitives

PIXELROOT relies on standard primitives: a collision-resistant hash (SHA-256 [19]) for commitments and PRNG seeding; Reed–Solomon codes [20] for error correction over the embedded payload; Merkle trees [21] for batching many commitments into a single on-chain transaction; and a public, append-only ledger [22], [23] as an immutable, publicly auditable timestamping witness. Large media artifacts are referenced—not stored—on-chain, optionally via content-addressed storage [24]. Hardware attestation of the signing environment follows established trusted-computing practice [12], [25].

III. RELATED WORK AND LITERATURE SURVEY

We organize prior art into five threads and then identify the gap PIXELROOT fills.

A. Deepfake generation and detection

Surveys by Verdoliva [2], Mirsky and Lee [26], and Croitoru *et al.* [1] chart the rapid progression from face-swap GANs to diffusion models and neural radiance fields across image, video, audio, and multimodal content. The recurring empirical finding is poor *cross-generator generalization*: detectors trained on one family of fakes collapse on another [1], [8]. The Deepfake-Eval-2024 in-the-wild benchmark quantifies this, reporting 45–50% AUC degradation versus curated academic sets [6]. The community increasingly frames generation vs. detection as an adversarial co-evolution in which detectors provide gradient signal to the next generator [7]. *Limitation:* detection is reactive, probabilistic, and structurally one step behind; it cannot offer affirmative, durable proof that a specific artifact is genuine.

B. Passive sensor forensics (PRNU)

PRNU-based source identification, introduced by Lukáš *et al.* [14] and extended to integrity verification by Chen *et al.* [15], estimates a camera fingerprint from residual noise and correlates it (e.g. via peak-to-correlation energy) with a questioned image; recent benchmarks track its accuracy on modern devices [27]. Broader integrity surveys situate PRNU among splicing, copy-move, and double-compression detectors [28], [29]. *Limitations:* (i) PRNU is *passive*—it requires a reference fingerprint and many images per device, and degrades on heavily compressed or low-resolution media [27]; (ii) it is vulnerable to the *fingerprint-copy* (PRNU-copy) attack, in which an adversary estimates a victim's fingerprint from public images and transplants it into a forged image. The triangle test and its successors detect naïve copies but are themselves defeated by improved, dispersed copy attacks [30]—a live cat-and-mouse. PIXELROOT differs by *actively* embedding a *chosen, keyed, error-corrected* payload bound to the OTP identity and a ledger timestamp, rather than relying on an involuntary, copyable noise pattern.

C. Active authentication: digital watermarking

Watermarking embeds a signal for authentication or copyright. Schemes are *robust* (survive processing, for copyright), *fragile* (break on any edit, for tamper localization), or *semi-fragile* (tolerate benign operations such as JPEG but break on malicious edits) [31], [32]. Classic methods embed in DCT/DWT/SVD coefficients, and the keyed spread-spectrum construction that PIXELROOT's carrier follows goes back to Cox *et al.* [33] and, for video, Hartung and Girod [34]; recent work uses learned encoders/decoders [35], and production systems now mark generated content at scale (SynthID [36], Video Seal [37], TrustMark [38]). The attack literature is equally mature: an adversary can estimate a mark from a single marked image and paste it onto another (the *copy attack* [39]), average many images to recover a fixed pattern [40], or regenerate an image with a diffusion model to remove pixel-level marks altogether [41], [42]. Watermarking inside the camera has also been proposed before: Blythe and Fridrich's *secure digital camera* embeds a lossless, keyed authentication mark at capture [43]. Video-domain schemes embed in H.264/HEVC transform coefficients or motion vectors and must contend with quantization and motion-compensation drift [44]–[46]. *Limitation:* watermarking alone provides no *public, time-anchored* record and no *hardware* root—a watermark embedded in software can be embedded by an attacker too—and a mark that encodes an identity can be copied with it [39], which PIXELROOT must and does address (§V-G). PIXELROOT treats its in-pixel payload as a *semi-fragile, hardware-originated* watermark whose trust is bootstrapped by on-chain notarization and OTP identity.

D. Provenance standards and secure capture

C2PA Content Credentials define a signed manifest of assertions (capture device, time, edit history) bound to the asset by a hard hash, verifiable offline via a certificate chain [9]. Hardware-backed signing has reached consumer devices—Google Pixel signs photos with keys in the Titan M2 secure element and an on-device timestamp authority [12]—Leica and Sony sign captures in cameras, and Samsung phones mark AI-edited images. Truepic's Controlled Capture verifies device integrity via attestation before signing on-device [10]; ProofMode captures sensor metadata, signs with per-capture keys, and registers hashes to public ledgers (OpenTimestamps/Bitcoin) [11]. *Limitations:* the binding lives in a *detachable manifest*. If a platform strips metadata (as many do) or transcodes the asset, the credential is lost and the bare pixels are again unverifiable; certificate compromise revokes trust wholesale [9]. PIXELROOT is complementary to C2PA—it can populate C2PA assertions—but additionally places a redundant proof *inside the pixels*, surviving manifest loss.

E. Blockchain-based media integrity

A large body of work anchors media hashes (exact and perceptual) to smart contracts, storing the bulk content off-chain in IPFS [24], [47]. Hasan and Salah propose using blockchain and smart contracts to trace deepfake video provenance back to a trusted original [48]; subsequent systems combine perceptual hashing, watermarking, and Ethereum/Polygon contracts for video copyright and integrity [49], [50]. *Limitations:* these systems hash media *after* capture, typically in software at upload time, leaving a tampering window and no hardware root; and exact hashes are destroyed by any re-encoding while perceptual hashes trade away cryptographic guarantees. PIXELROOT narrows the window to the shutter event and adds a hardware-anchored in-pixel binding the ledger alone cannot provide.

F. Gap analysis and precise novelty

Table I summarizes the landscape. Each ingredient exists in isolation: watermarking embeds recoverable signals in pixels; PRNU ties images to a physical sensor; C2PA provides hardware-backed signed provenance; and prior systems anchor media hashes to a blockchain [48]; even in-camera watermarking has precedent [43]. Our claim is therefore *not* that any single component is new. The specific, defensible novelty of PIXELROOT is their *co-design*: a **(i) hardware-rooted, key-protected, error-corrected signal embedded by the sensor's own gain path**—so the binding survives metadata stripping and is rooted in silicon rather than a software key—that is **(ii) bound at the shutter to a public-ledger commitment**, yielding two independent failure-disjoint bindings, **(iii) realized through a *feasibility-grounded* per-column/per-region gain primitive** we ask manufacturers to standardize rather than the per-pixel control prior "sensor-embedding" sketches assumed, and **(iv) bound to a sensor key certified and revocable on the same public ledger**, through a signed, content-bound record that makes the in-pixel identity non-transferable. No prior approach occupies the intersection of hardware root, metadata-strip survival, and public real-time notarization. We claim contributions at the level of architecture, the concrete hardware ask, the measured robustness of the carrier, and the privacy design—not the invention of watermarking or blockchains.

TABLE I
WHERE PRIOR ART SITS, AND THE GAP PIXELROOT TARGETS.

Approach	HW root	Surv. meta-strip	Public ledger	Affirm. proof	Real-time
Deepfake detectors [1]	✗	—	✗	✗	✓
PRNU forensics [14]	part.	✓	✗	part.	✗
Watermarking [31]	✗	✓	✗	part.	✓
C2PA / Truepic [9], [10]	✓	part.†	opt.	✓	✓
Blockchain hashing [48]	✗	✗	✓	✓	part.
PIXELROOT (this work)	✓	✓	✓	✓	✓

HW root = hardware/sensor root of trust; Surv. meta-strip = survives metadata removal; Affirm. proof = affirmative proof of authenticity.
†C2PA 2.x can recover a stripped manifest through a registered watermark or fingerprint (soft binding) when one is used.

IV. THREAT MODEL AND DESIGN GOALS

A. Assumptions

We assume: (A1) the sensor's OTP identity and gain registers are written at manufacture and are not feasibly reprogrammable in the field without specialized equipment; (A2) the capture-time signing/notarization environment is attested (e.g. via a secure element), as in current hardware-backed C2PA deployments [10], [12]; (A3) the blockchain is append-only and its history is economically infeasible to rewrite within the relevant timeframe [22]; (A4) standard cryptographic primitives are secure [19], [51]; (A5) each sensor generates its signing key on-die at manufacture and the private key cannot be exported without an invasive attack on that physical device; manufacturers admitted to the registry certify only keys produced on their lines.

B. Adversary

The adversary may: forge or edit file metadata; generate a deepfake or Photoshop a real image; re-encode/transcode/compress media; re-photograph a screen (analog "re-capture"); attempt to estimate and transplant a victim's sensor fingerprint (PRNU-copy [30]); and replay or back-date claims; read any published capture record; estimate and copy an in-pixel mark from images it obtains (with or without the original); and extract the key of a device it physically possesses. The adversary cannot economically clone a specific physical CMOS die, extract keys from devices it does not hold, or rewrite ledger history.

C. Design goals

G1 Affirmative authenticity: produce verifiable evidence that media originated from a genuine sensor at a specific time/place. **G2 Hardware root:** bind the proof to physical sensor identity, not to software-only secrets. **G3 Metadata-independence:** retain evidentiary value even if the file container/metadata is stripped. **G4 Public verifiability:** any third party (platform, court, viewer) can verify a capture from public data—the ledger, the published record, and the OEM's carrier key—without contacting the originator or possessing the camera (§X). **G5 Minimal capture latency:** submit the commitment at capture, so the sealing delay is the batch interval plus the chain's block time (seconds on low-latency chains and rollups); we measure gas but leave public-network latency to future work (F4). **G6 Imperceptibility:** embedding must not visibly degrade the image. **G7 Standardizability:** be deployable as a cross-OEM interoperability standard.

V. THE PIXELROOT ARCHITECTURE

PIXELROOT comprises a capture-time *embed-and-notarize* pipeline and a *verification* pipeline. Fig. 1 shows the end-to-end architecture: the upper band is the on-device capture path that produces the two bindings, and the lower band is the third-party verification path that consumes them.

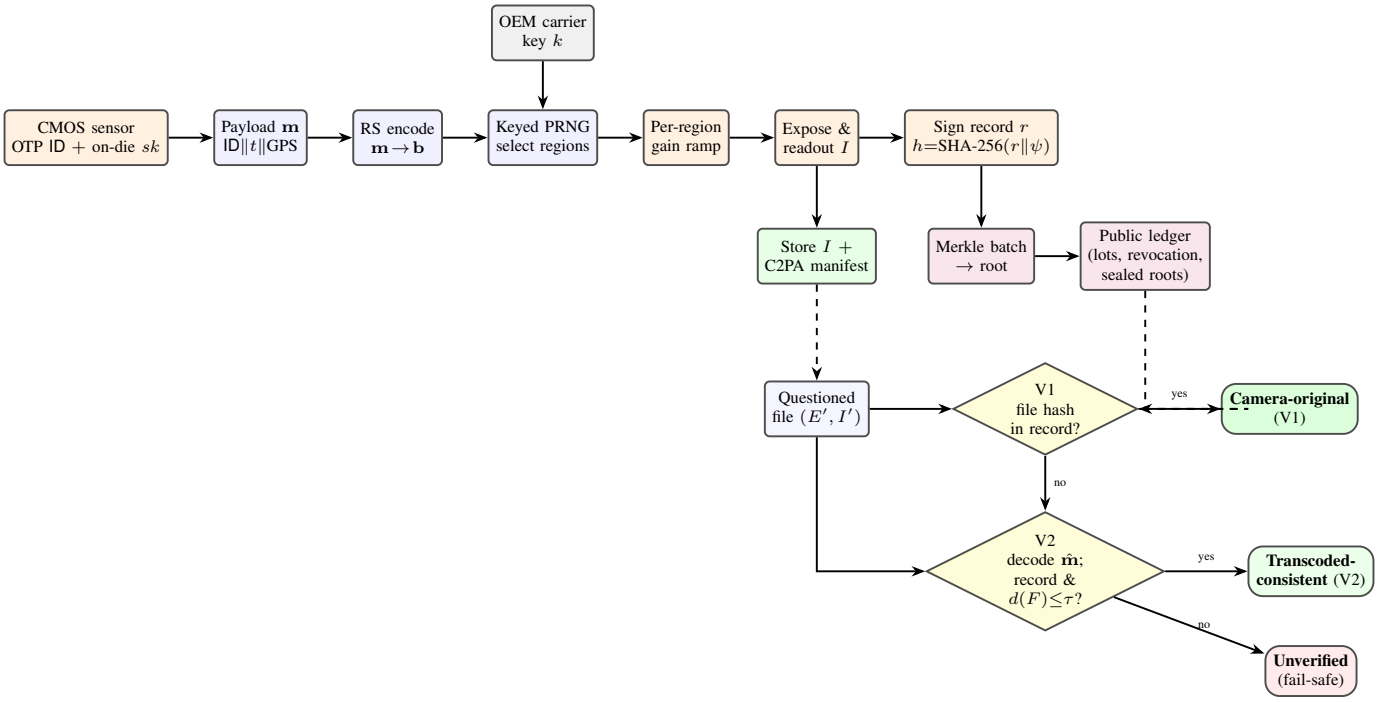


Fig. 1. PIXELROOT architecture. *Top:* the capture path emits a hard binding (the hash of a sensor-signed capture record, sealed on the public ledger) and a soft in-pixel binding (the keyed, error-corrected gain pattern carried inside I); the same ledger certifies and revokes sensor keys. *Bottom:* verification checks V1 (the file's hash is committed in a valid record) and V2 (the decoded payload resolves to a valid record whose content fingerprint matches) and returns the strongest supported level, failing safe to "unverified".

A. Hardware feasibility: the gain-control spectrum

A central objection to any "sensor-level embedding" proposal is that commercial CMOS sensors do *not* expose arbitrary per-pixel gain. PIXELROOT does not require it. We organize sensor gain controllability into a spectrum and target the realistic middle of it:

- **Global gain (today).** A single ISO/analog-gain register per frame. Too coarse to carry a spatial payload.
- **Per-column gain (today).** Column-parallel readout places a programmable-gain amplifier per column; per-column gain trim is routinely used for column fixed-pattern-noise correction. A per-column gain *profile* directly imprints horizontal low-frequency content—the shape of PIXELROOT's carrier. A static profile, however, applies the same gain to every row of a column and so gives every block in a block-column the same chip sign; the block-keyed layout additionally needs the column gains to be re-programmed as readout advances, i.e. at row (line) rate in a rolling-shutter sensor, which column front-ends must be shown to support.
- **Per-region / tiled gain (emerging).** Dual- and triple-conversion-gain and HDR-zone sensors already switch gain over regions by toggling floating-diffusion (FD) capacitance [52]–[54]; a coarse $K \times K$ gain tile map is a small, standardizable register extension.
- **Per-pixel gain (manufacturable, not yet productized).** Arbitrary per-photosite gain is not exposed by today's commodity parts, but it is *not* a physics barrier—it is an integration-cost choice, and three process technologies already in volume manufacturing make it achievable (§V-B). PIXELROOT therefore treats per-pixel control as the apex target to standardize, not an impossibility to avoid.

The PIXELROOT ask (tiered). We ask manufacturers for a *provenance-write* capability, achievable at three increasing cost points: (1) a programmable *per-column* gain vector and a coarse *per-region* tile map addressable at capture—realizable on conventional sensors today; and, as the apex, (2) *per-pixel* gain via the manufacturing routes of §V-B. All tiers reuse the OTP identity and attested clock that hardware-backed C2PA pipelines already ship [10], [12]. PIXELROOT's robust carrier needs only a *low-frequency gain ramp over an image region* (§V-E), which per-region control realizes, as does per-column control re-programmed at line rate; per-pixel control is strictly more general—it *subsumes* per-column and per-region patterns and additionally enables a fine-grained tamper-localization layer (§V-B).

Software-ISP fallback (legacy). Devices without any gain capability can apply the identical modulation in the attested image-signal processor (ISP) immediately after readout. This preserves the in-pixel binding and notarization but *weakens the hardware root*: the modulation is imposed in firmware rather than the photo-electron domain, so it is only as trustworthy as the attested execution environment (comparable to today's secure-capture apps). We label such captures at a lower assurance tier (§X) and never conflate them with sensor-rooted captures.

B. Manufacturing per-pixel gain: a feasibility analysis

Because a reviewer (and an OEM) will ask *exactly how* a fab would build per-pixel intensity control, we survey the silicon options and their cost. Three complementary, already-shipping process technologies make per-pixel gain achievable; PIXELROOT can use any of them, and they can be combined.

a) (M1) Per-pixel conversion-gain selection (FD-capacitance): Conversion gain—the volts-per-electron set at the floating-diffusion node—is already made *switchable* per pixel in dual-/triple-conversion-gain (DCG/TCG) pixels by adding an in-pixel MOSFET that connects or isolates extra FD capacitance [52], [53], [55]. Production mobile sensors switch among 2–3 discrete CG states per pixel (Samsung's $16{:}4{:}1$ triple-CG sensor reaches 82.4 dB single-exposure HDR at a $1.4\,\mu$m pitch [54]). *The provenance change is small*: drive the existing DCG/TCG select line from the keyed PRNG rather than only from a scene-adaptive HDR controller, so each carrier pixel is exposed at a chosen CG state. This imprints the payload *in the analog photo-electron domain* (the strongest root) and reuses transistors that millions of sensors already ship; its limitation is quantization to a few discrete gain levels, which is ample for 1–2 bits/pixel under PIXELROOT's spread-spectrum coding (Fig. 3).

b) (M2) Per-pixel exposure/charge modulation (coded-exposure pixels): A second, demonstrated route adds a per-pixel switch plus a one-bit code memory (e.g. in-pixel SRAM) so each pixel's integration can be gated independently—the basis of pixel-wise programmable exposure and coded-exposure imagers [56]–[59]. Modulating integration time is equivalent to modulating effective gain; a tiny keyed exposure delta on carrier pixels embeds the payload without any optical modulator. These designs have been fabricated in standard CMOS, confirming manufacturability; the cost is the extra in-pixel transistors/SRAM, which enlarge pixel pitch unless absorbed by stacking (M3) (Fig. 4).

c) (M3) Stacked per-pixel processing (3D integration + hybrid bonding): The decisive enabler is 3D stacking. Backside-illuminated (BSI) sensors put the photodiode on one wafer and move all circuitry to wafers bonded beneath it; with *Cu–Cu hybrid bonding* the two wafers are joined by per-pixel vertical copper interconnects with no bumps. Sony shipped the first hybrid-bonded BI-CIS in 2016 and has since driven the interconnect pitch below $2\,\mu$m [60], [61]; research bonders reach a 400 nm pitch with $\sim 10^6$ interconnects per mm^2 [62]. This makes *digital pixel sensors* (DPS) practical: each pixel gets its own ADC and in-pixel memory on a stacked tier—e.g. a 3-layer DPS with a 10-bit in-pixel SRAM and on-die ISP [63]–[65], and three-wafer sensors that place a DNN/ISP tier beneath the pixels [66]. Once a per-pixel ADC, memory, and logic tier exist, *per-pixel gain is essentially free*: the stacked logic multiplies each pixel's digitized value by a per-pixel factor read from the keyed code memory *inside the attested sensor die*, before any output leaves the chip. This is stronger than the ISP fallback (the modulation never leaves trusted silicon) though still a digital—not photo-electron—operation; combining M3 with M1 yields both a robust digital channel and an analog-domain root (Fig. 5).

d) Cost and trade-offs: The honest costs are area, pixel pitch, yield, and power: extra in-pixel devices (M1/M2) compete with the photodiode for area unless moved to a stacked tier (M3), and stacking adds bonding-yield and thermal-management constraints [60]. Focal-plane sensor-processors that put a full processing element at every pixel pay for it with large pitch and low resolution [59], [67]; PIXELROOT avoids that extreme because it needs only a small per-pixel gain register, not a general processor. *A caution on image quality:* writing a keyed gain profile interacts with the very subsystems built to *remove* spatial gain variation—fixed-pattern-noise and lens-shading correction, auto-exposure, HDR fusion, and demosaicing—so the modulation must be small, low-frequency, and either applied before or jointly calibrated with these blocks; quantifying that interaction (and the residual fixed-pattern-noise budget it consumes) is a silicon-validation item we flag as future work (F5), not a solved problem. *Net assessment:* the building blocks of per-pixel provenance gain—DCG/TCG selection, coded exposure, and stacked DPS—are already in high-volume manufacturing, so the primitive is an *evolutionary* increment rather than new device physics; what remains is a standardization and image-quality co-design effort, not a physics breakthrough. We therefore recommend OEMs explore exposing a keyed per-pixel (or, minimally, per-column) gain-write path as a capture primitive.

e) What per-pixel buys PIXELROOT*: a two-layer mark:* Per-pixel control lets PIXELROOT carry *two* bindings in the pixels at once: (i) the robust, low-frequency spread-spectrum payload of §V-E, painted as a smooth per-region ramp, which carries the 128-bit identity and survives JPEG and rescaling (measured in §IX-C); and (ii) an optional high-resolution per-pixel pattern keyed to the payload that acts as a spatial *integrity map*. The robust layer answers "is this from a genuine device?"; the per-pixel layer answers "*which pixels* were altered?" by localizing splices at block granularity—a capability only per-pixel gain unlocks. On real images with real splices this layer localizes tampering at ROC-AUC 0.97 (block TPR/FPR $0.91/0.04$, 32 px granularity), and degrades gracefully under recompression (AUC 0.95 at JPEG $Q90$; §IX-C), so it is intended for lightly-processed forensic media rather than as the primary identity channel.

C. Provenance payload

The payload $\mathbf{m}$ is a fixed-width record (PIXELROOT v2: 128 bits):

$$\mathbf{m} = \underbrace{\mathrm{ID}}_{48} \,\|\, \underbrace{t}_{32} \,\|\, \underbrace{(\phi, \lambda)}_{48}, \tag{1}$$

where ID is, *by default*, a per-device *pseudonymous* attestation identifier issued by the secure element (a rotating 48-bit token, not a global serial number, and unlinkable to the physical device without OEM cooperation), t is a Unix timestamp from an attested RTC/NTP source, and (ϕ, λ) is an *opt-in*, off-by-default quantized latitude/longitude (24 bits per coordinate, $\approx 2 \times 10^{-5}$ degree resolution). To protect the journalists and activists PIXELROOT aims to serve, the recommended default is pseudonymous ID with *no* location; precise location, when

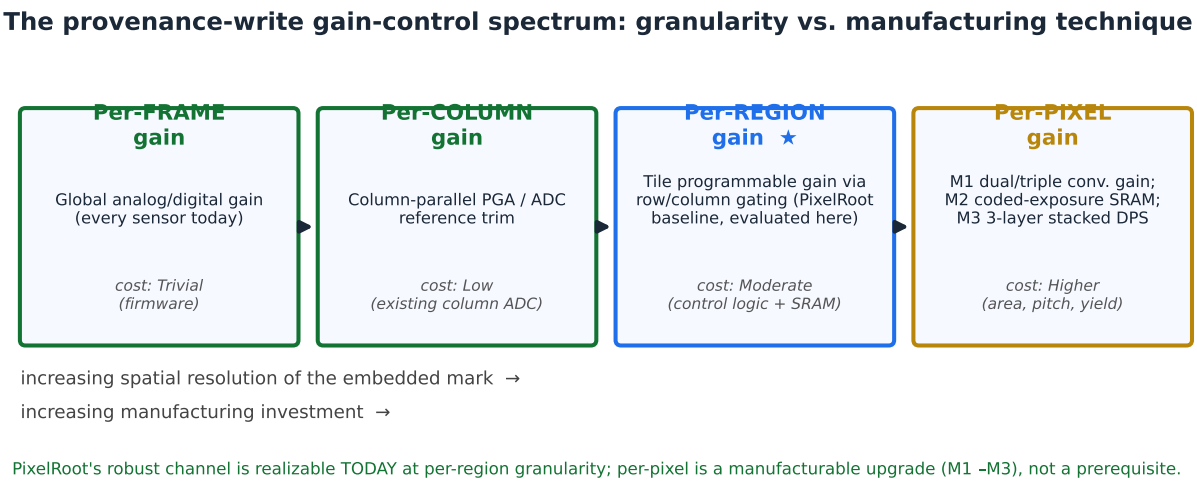


Fig. 2. The provenance-write gain-control spectrum. PIXELROOT's robust channel is realizable *today* at per-region granularity (column-parallel front-ends); per-pixel gain is a manufacturable upgrade via M1–M3, not a prerequisite.

enabled, may be coarsened or replaced by a salted commitment disclosed only under subpoena (§X).

D. Keyed region selection and spread-spectrum coding

PIXELROOT partitions the frame into 8×8 blocks aligned with the JPEG grid and treats *blocks*, not single pixels, as carriers. The carrier layout is derived from the OEM *carrier key* k,

$$s = \text{SHA-256}(k), \tag{2}$$

which keys a cryptographic PRNG (a stream/CTR-mode keyed generator) that draws a permutation of the N_B blocks of the frame, so the layout is fixed for a given key and frame size. The layout deliberately does *not* depend on the payload $\mathbf{m}$: a verifier that holds only the pixels must be able to locate the carriers and *read* $\mathbf{m}$, which is what lets the in-pixel binding survive metadata stripping. (The released prototype seeds NumPy's legacy generator with a 32-bit reduction of s; a deployment uses the full-width keyed PRNG.) For each coded bit the PRNG draws R distinct carrier blocks without repetition and assigns each an antipodal *chip* $c_j \in \{\pm1\}$ (spread spectrum). The 128 payload bits are first protected by error correction (a Reed–Solomon code over $\mathrm{GF}(2^8)$ [20]); the repetition factor R gives each bit redundancy across many spatially dispersed blocks. A party without k that *reads* a marked image obtains a random bit string (Prop. 2). The layout is, however, the same for every frame of a given size, and the detector's statistic is computable from public information, so an attacker *can* estimate the mark from marked images and copy or subtract it (§IX-D). PIXELROOT therefore does not rely on the secrecy of k for authenticity: the in-pixel payload is a *pointer* whose meaning is fixed by a sensor-signed, content-bound record on the ledger (§V-G). Copying the pointer onto other content yields a verdict of UNVERIFIED, and removing it yields the same fail-safe result.

E. Region-gain (spread-spectrum) embedding

The single-pixel, 2% gain perturbation of naïve designs is a poor carrier: an isolated spike is high-frequency energy, precisely what JPEG discards, and it cannot be read back reliably against textured scene content. PIXELROOT instead embeds each bit as a *low-frequency gain ramp over its carrier blocks*. Let $\Phi_{u,v}(x,y) = \cos\frac{(2x+1)u\pi}{16}\cos\frac{(2y+1)v\pi}{16}$ be a low-order 8×8 DCT basis (we use a low horizontal frequency, $(u,v)=(0,2)$, whose shape a per-column gain profile produces). For a carrier block B_j with chip c_j carrying bit $b\in\{0,1\}$ with sign $\sigma = 2b-1$, the sensor adds

$$\Delta_j(x,y) \;=\; A\,\sigma\,c_j\,\Phi_{u,v}(x,y), \tag{3}$$

i.e. a smooth gain ramp of peak amplitude A gray levels (about 3% of mid-gray at $A=4$). Equivalently, the chosen low-frequency DCT coefficient of B_j is shifted by $\pm A$. This reconciles two domains: the modulation is applied as a *spatial* per-region gain (hardware-realizable, §V-A) but lives in a *low-frequency DCT* band (JPEG-survivable). Because $\Phi_{u,v}$ for $(u,v)\neq(0,0)$ is zero-mean over the block, the embedding leaves block brightness essentially unchanged and is imperceptible (measured mean PSNR 38.2 dB at the $A=4$ operating point and 41.2 dB at $A=3$, §IX).

a) Correlation read-back: A questioned image I' is decoded by *projecting* each carrier block onto the same basis—no per-pixel thresholding and, crucially, no estimate of the block's "original" brightness is needed, because the AC basis is orthogonal to the (content-dominated) DC term. For block B_j,

$$\rho_j \;=\; \frac{\langle I'_{B_j},\,\Phi_{u,v}\rangle}{\langle \Phi_{u,v},\,\Phi_{u,v}\rangle} \;\approx\; A\,\sigma\,c_j \;+\; \eta_j, \tag{4}$$

where η_j is the host-content and noise term. Because neighbouring blocks of natural images have similar $(0,2)$ coefficients while the chips are independent and zero-mean, the detector first high-passes the field of projections, $\tilde{\rho}_j = \rho_j - \overline{\rho}_{\mathcal{N}(j)}$,

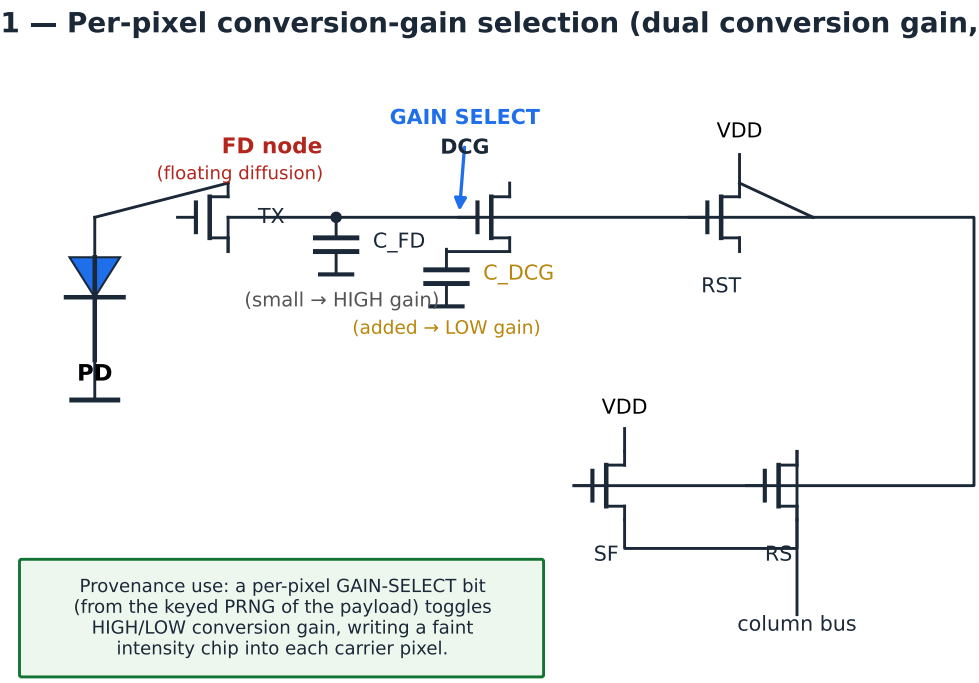


Fig. 3. M1: a 4T pixel with a per-pixel dual-conversion-gain (DCG) transistor. A keyed GAIN-SELECT bit switches the extra FD capacitance C_{DCG} in/out, toggling HIGH/LOW conversion gain to imprint an analog-domain chip.

where $\overline{\rho}_{\mathcal{N}(j)}$ is the mean over the 5×5 blocks around B_j; this removes most of the slowly varying host term. De-spreading and combining the R blocks of a bit,

$$D \;=\; \sum_{j=1}^{R} c_j\,\tilde{\rho}_j \;\approx\; R\,A\,\sigma \;+\; \textstyle\sum_j c_j\tilde{\eta}_j, \qquad \hat{b}=\mathbf{1}[D>0], \tag{5}$$

so the payload signal grows as RA while the random chips make the host terms average toward zero ($\mathrm{std}\propto\sqrt{R}$): the detection signal-to-noise ratio improves as $\sqrt{R}$. The per-bit magnitude $|D|$ is the soft confidence passed to the error-correction decoder. This is a textbook blind, spread-spectrum watermark detector, and it is what keeps the measured raw BER in §IX near 2% through aggressive JPEG—low enough for the error-correcting code to recover the payload.

F. Commitment and notarization

Immediately after readout, the sensor forms and signs a *capture record* (§V-G) that commits, among other fields, to the raw frame $\mathcal{H}(I)$, to the camera's own encoded file $\mathcal{H}(E(I))$, and to the payload $\mathbf{m}$. The hard binding is the record hash

$$h=\mathrm{SHA\text{-}256}\big(\,r\,\|\,\psi\,\big), \tag{6}$$

where r is the record and ψ the sensor's signature on it. For scale, many record hashes are aggregated into a Merkle tree [21] and only the root is written on-chain, amortizing gas cost; the contract records the block timestamp, which serves as the public "proof of existence" [11]. If connectivity is briefly unavailable, h is queued in secure storage and submitted on reconnect, with the latency recorded.

Algorithm 1 PIXELROOT capture-time embed, sign, and notarize

Require: carrier key k, amplitude A, repetition R, basis $\Phi_{u,v}$; on-die key sk with certified $pk\in$ lot Λ

$\beta \leftarrow$ (number, hash) of a recent block {capture-time lower bound}
ID $\leftarrow$ readOTP() {48-bit pseudonymous handle}
$t \leftarrow$ attestedClock(); $(\phi,\lambda_{\mathrm{geo}}) \leftarrow$ gps()
$\mathbf{m} \leftarrow \mathrm{ID}\,\|\,t\,\|\,\mathrm{quant}(\phi,\lambda_{\mathrm{geo}})$ {128 bits}
$\mathbf{b} \leftarrow$ ECC-Encode($\mathbf{m}$) {RS, coded bits $b_1\ldots b_{n_c}$}
prng $\leftarrow$ CSPRNG(SHA-256(k))
for all coded bit b_i, $\sigma_i \leftarrow 2b_i-1$ **do**
  draw R distinct blocks $\{B_j\}$, chips $c_j\in\{\pm1\}$ via prng
  for all (B_j,c_j) **do**
    set per-region gain ramp on B_j: add $A\,\sigma_i c_j\,\Phi_{u,v}$
  end for
end for
$I \leftarrow$ expose_and_readout(); $E \leftarrow$ encode(I)
$r \leftarrow (pk,\mu,\Lambda,\mathbf{m},\mathcal{H}(I),\mathcal{H}(E),F(I),\beta,c)$; $\psi \leftarrow \mathrm{Sign}_{sk}(r)$
$h \leftarrow \mathrm{SHA\text{-}256}(r\,\|\,\psi)$; enqueue h for Merkle batch
on flush: $root \leftarrow \mathrm{Merkle}(\{h\})$; register($root$)
store E with (r,ψ,π) in a C2PA manifest; publish (r,ψ,π) to the record store when E is published

Step-by-step: (1) **Anchor**: note a recent block β. (2) **Acquire identity & context**: the 48-bit OTP handle, the attested clock t, and optional location. (3) **Form payload $\mathbf{m}$** (128 b) and (4) **channel-code** it with RS over $\mathrm{GF}(2^8)$ into n_c coded

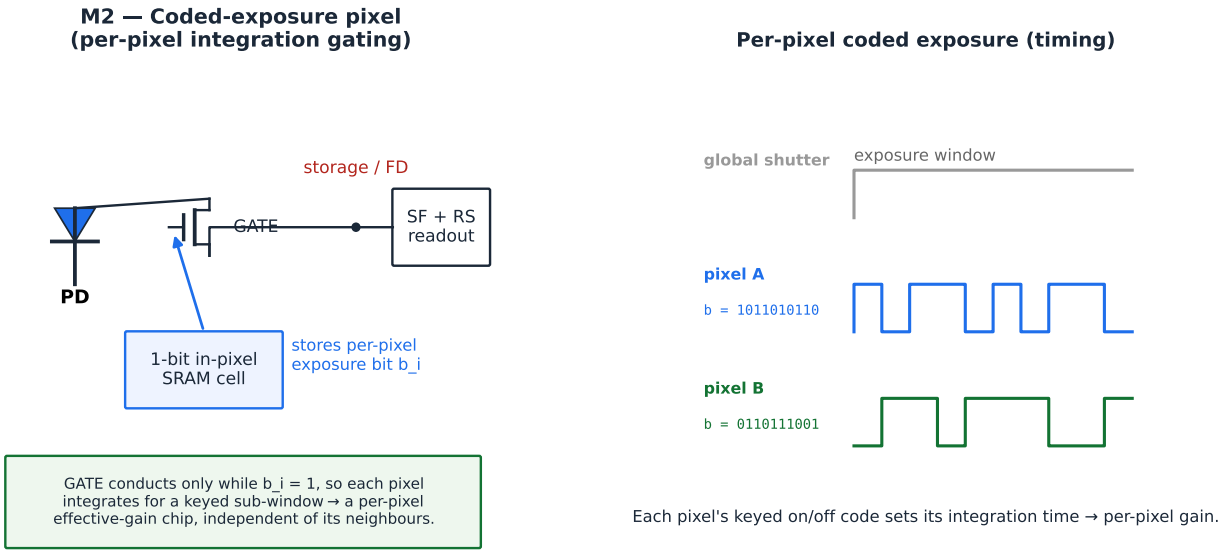


Fig. 4. M2: a coded-exposure pixel. A 1-bit in-pixel SRAM gates integration, so each pixel's keyed on/off code sets its effective gain independently of its neighbours (timing, right).

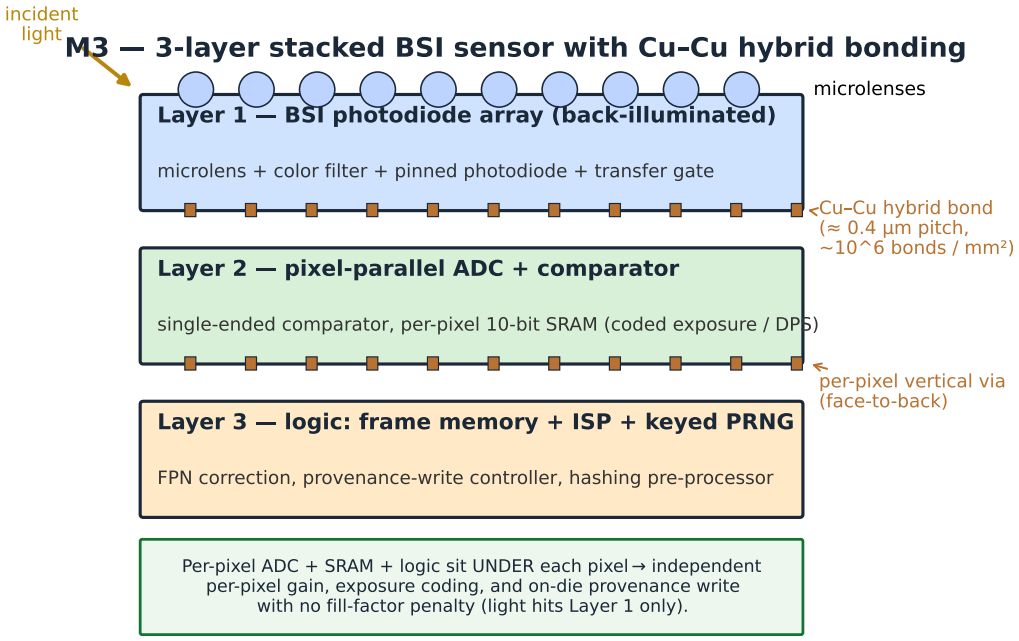


Fig. 5. M3: a 3-layer stacked BSI sensor joined by Cu–Cu hybrid bonding ($\approx 0.4\,\mu$m pitch, $\sim 10^6$ interconnects/mm^2). A per-pixel ADC, SRAM, and a logic tier (frame memory, ISP, keyed PRNG/provenance-write) sit under each pixel with no fill-factor penalty.

bits. (5) **Select carriers**: the keyed PRNG assigns each coded bit R distinct 8×8 blocks with antipodal chips. (6) **Modulate**: the sensor adds a low-frequency gain ramp $A\sigma_i c_j \Phi_{u,v}$ to each carrier block *during* exposure (Eq. 3), in the photo-electron domain (per-region gain) rather than in software. (7) **Read out** the frame I and encode the file E. (8) **Sign**: the sensor signs the capture record with its on-die key. (9) **Notarize**: the record hash is batched into a Merkle tree and the root is registered on-chain in one transaction. (10) **Persist** the record, signature and inclusion proof π in a C2PA manifest, and publish them to the record store when the image is published. The manifest is a convenience: V2 needs only the pixels, the record store, and the ledger.

G. Device identity and content-bound records

The OTP identifier inside $\mathbf{m}$ names a sensor, but it proves nothing: it is a public number, and the in-pixel mark that carries it can be estimated from a single shared image and pasted onto other content without the carrier key (the classic copy attack [39]; §IX-D measures a 78% success rate at the watermark level). Identity must therefore rest on a secret that only the sensor holds and that the ledger can vouch for.

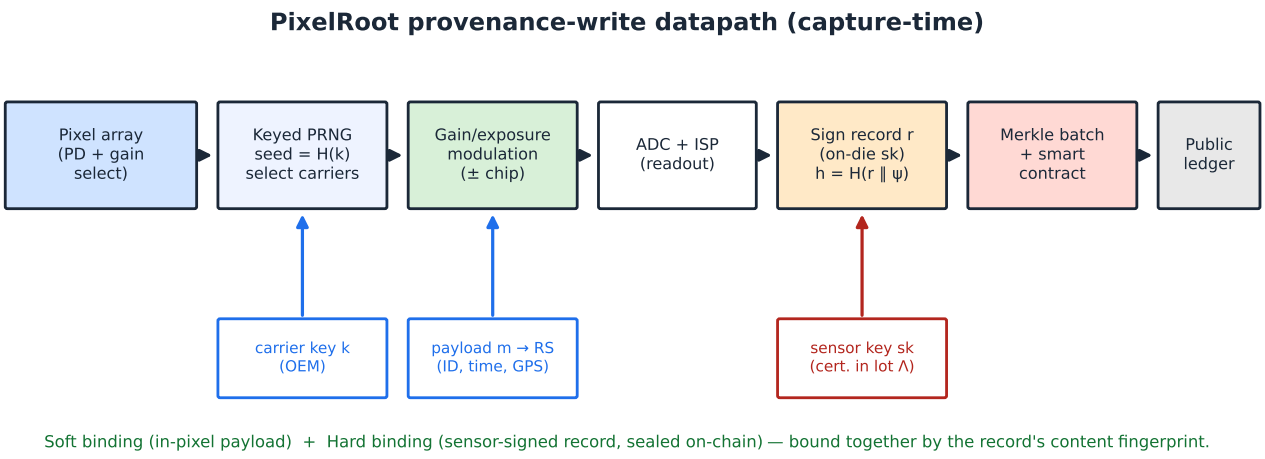


Fig. 6. The PIXELROOT provenance-write datapath at capture time. The keyed PRNG (seed $\mathcal{H}(k)$) selects carriers; the gain/exposure hardware applies the RS-coded payload as $\pm$chips; after readout the sensor signs a capture record with its on-die key, and the record hash is batched into a Merkle tree sealed on the public ledger. The in-pixel payload is the soft binding (survives transcoding); the sensor-signed record on the ledger is the hard binding; the record's content fingerprint ties the two together (Props. 1, 4).

PIXELROOT adds four elements, all anchored on the same blockchain as the captures.

a) An on-die sensor key: At manufacture each sensor generates an ECDSA P-256 key pair [51] inside the die—in a secure element on the stacked logic tier (M3, §V-B) or derived from a physically unclonable function—so the private key sk never leaves the sensor, not even to its manufacturer. The OTP identifier becomes a pseudonymous handle; the public key pk is the credential. Because every key is unique to one die, physically extracting a key compromises one device, not a fleet.

b) Certification and revocation on the ledger: Manufacturers are admitted to the registry by its governance (in a standard, a consortium multisignature). A manufacturer μ certifies a production lot by registering the Merkle root Λ of $\{\text{SHA-256}(pk_i)\}$; each sensor stores its inclusion path, so a lot of 10^5 sensors costs one transaction. Lots are keyed by the pair (μ, Λ), so no other party can claim, or revoke within, a manufacturer's lot, even by registering the same root first. A manufacturer can revoke one of its sensor keys (a stolen or physically attacked device), and governance can revoke a manufacturer. Both are *time-scoped*: the ledger records the revocation time, captures sealed before it remain valid, and later ones fail. Revocation is enforced by the contract's own verification call, not left to validator policy—the gap that let revoked certificates keep validating after the Nikon incident [13].

c) A signed, content-bound capture record: After readout the sensor forms

$$r = \big(pk,\ \mu,\ \Lambda,\ \mathbf{m},\ \mathcal{H}(I),\ \mathcal{H}(E(I)),\ F(I),\ \beta,\ c\big) \tag{7}$$

and signs it, $\psi = \text{Sign}_{sk}(r)$, where c is a monotonic capture counter kept in the sensor. Committing $\mathcal{H}(E(I))$, the hash of the camera's own encoded file, makes the hard binding checkable on the file people actually receive; $\mathcal{H}(I)$ keeps raw-domain forensics possible. $F(I)$ is a robust content fingerprint of the frame's *interior*: a border of $M = 32$ px—at least the largest translational offset the decoder searches—is excluded, the interior is divided into 8×8 tiles, each tile's luma is box-downsampled to 16×16 cells, and the 8×8 lowest DCT coefficients of each tile, taken relative to the interior's mean (so the DC term encodes the tile's brightness), are kept (8,448 bytes at int16). The fingerprint distance is the largest per-tile relative error,

$$d(F, F') = \max_{\ell} \frac{\|f_\ell - f'_\ell\|_2}{\|f_\ell\|_2 + \varphi}, \tag{8}$$

with an energy floor φ so that flat tiles do not dominate. Because no tile is ever skipped and the excluded border is wider than any accepted shift, a crop cannot hide an edit inside the fingerprinted region. Compression, noise, mild filtering, rescaling and photometric edits leave every tile's coarse structure nearly unchanged; different content, or a foreign region large enough to change a tile, does not. The record ($\approx$18 KB as hex JSON) is published by the capturing party, the OEM, or a content-addressed store when the image is published; only its hash reaches the chain. The record store is untrusted: a lookup by payload may return several records (bursts captured within one second share $\mathbf{m}$), and every candidate is checked against the ledger.

d) What a transcoded verdict means: A fingerprint that survives JPEG $Q10$ must ignore detail below its resolution, and an adversary who holds the original can re-impose the original's cell means on an edited image (§IX-D). V2 therefore certifies *derivation*, not pixel integrity: TRANSCODED-CONSISTENT means that the questioned image carries a payload whose sensor-signed record describes the same scene at the fingerprint's resolution, and the verifier returns that record—and, when the capturing party publishes it, the original camera file—so that a viewer can compare the two. Only CAMERA-ORIGINAL (V1) certifies exact pixels. Copy attacks across scenes, large splices, and fabricated content are rejected;

local edits within one capture are surfaced by comparison with the original, not excluded by V2.

e) A capture-time bracket: β is the number and hash of a block the device observed before exposure. The contract records both the time and the block number at which each batch is sealed; a record whose anchor block is later than its sealing block, or whose anchor hash is not the canonical hash of that block, is rejected, and a valid anchor proves the record was formed after block β—a *lower* bound on capture time that complements the upper bound of Prop. 1 (Prop. 5).

f) Why not ask the manufacturer to sign each capture: An online, per-capture signature by the manufacturer would add no security over the sensor's own certified key, and it would cost a great deal: capture would fail offline (conflict zones, aircraft, courtrooms); the manufacturer would learn who captured what, when and where; and a breached, coerced, or defunct manufacturer would stop or forge its whole fleet's provenance. PIXELROOT therefore confines the manufacturer to certifying keys once and revoking them on the ledger; an optional manufacturer co-signature of batch roots can serve as a higher assurance tier, never as a requirement.

g) The role of the carrier key: With identity carried by the signed record, the carrier key k protects the robustness of the channel, not its authenticity: a forger who knows k can read or copy a payload but still cannot produce a record that a certified sensor signed for the forged content. An OEM may therefore publish its carrier key, which makes V2 verifiable by anyone—closing the gap between the publicly verifiable hard binding and the key-holder-verified soft binding (goal G4).

H. Smart-contract registry

On-chain state is intentionally minimal and holds only ledger facts: admitted manufacturers, certified lots of sensor keys, revocation times, and the block time at which each batch root was sealed. Every Merkle tree (lots and capture batches) uses the same domain-separated construction, $\text{leaf} = \text{SHA-256}(\texttt{0x00} \| x)$ and $\text{node} = \text{SHA-256}(\texttt{0x01} \| \min(a,b) \| \max(a,b))$, with an odd node promoted rather than duplicated; the prefix stops an internal node from being presented as a leaf, and promotion rules out the duplicate-last-node mutation. Batching N captures under one root makes per-image gas $O(1)$ amortized, and inclusion is proven with a $\lceil \log_2 N \rceil$-length path. Sealing is deliberately permissionless—any device, app, or batcher may submit, so no party can censor a capture—and the submitter carries no trust: authenticity comes from the sensor signature inside each record, checked against the certified lots. Listing 1 shows the notarization and verification calls; The full contract, an 18-case test suite, and the measured gas (§IX-D) are released.

Listing 1. PIXELROOTRegistry (v2): permissionless batch sealing and the verification calls (excerpt).

```
    /// @notice Seal a Merkle root of capture-record hashes. Deliberately
    /// permissionless (any device, app or batcher may submit, so no
    /// party can censor a capture); `submitter` and `leafCount` are
    /// informational and carry no trust. Authenticity comes from the
    /// sensor signature inside each record, checked against the lots.
    function register(bytes32 root, uint256 leafCount) external {
        require(root != bytes32(0), "empty root");
        require(seals[root].time == 0, "already sealed");
        seals[root] = Seal(uint64(block.timestamp), uint64(block.number));
        emit RootRegistered(root, msg.sender, uint64(block.timestamp), uint64(block.number), leafCount);
    }

    // ---------- verification ----------

    function sealedAt(bytes32 root) public view returns (uint64) {
        return seals[root].time;
    }

    function sealedBlock(bytes32 root) public view returns (uint64) {
        return seals[root].blockNumber;
    }

    function leafHash(bytes32 item) public pure returns (bytes32) {
        return sha256(abi.encodePacked(bytes1(0x00), item));
    }

    function lotKey(address manufacturer, bytes32 lotRoot) public pure returns (bytes32) {
        return keccak256(abi.encodePacked(manufacturer, lotRoot));
    }

    /// @notice True iff `recordHash` is in a sealed capture batch `root`.
    function verifyInclusion(bytes32 root, bytes32 recordHash, bytes32[] calldata proof)
        external view returns (bool included, uint64 timestamp)
    {
        timestamp = seals[root].time;
        if (timestamp == 0) return (false, 0);
        return (_verify(root, recordHash, proof), timestamp);
    }

    /// @notice True iff `manufacturer` certified the sensor key in `lotRoot` no
    /// later than time `t` and neither the key nor the manufacturer was
    /// revoked by `t`.
    function deviceValidAt(
        address manufacturer,
        bytes32 lotRoot,
        bytes32 deviceKeyHash,
        bytes32[] calldata proof,
        uint64 t
    ) public view returns (bool) {
        bytes32 key = lotKey(manufacturer, lotRoot);
        Lot memory lot = lots[key];
        if (lot.sealedAt == 0 || lot.sealedAt > t) return false;
        uint64 mRev = manufacturerRevokedAt[manufacturer];
        if (mRev != 0 && mRev <= t) return false;
        uint64 dRev = deviceRevokedAt[keccak256(abi.encodePacked(key, deviceKeyHash))];
        if (dRev != 0 && dRev <= t) return false;
        return _verify(lotRoot, deviceKeyHash, proof);
    }

    /// @notice One-call ledger check for a capture: the record is sealed (at
    /// `timestamp`, in block `blockNumber`, which the verifier compares
    /// with the record's anchor block), and the signing sensor was valid
    /// at the sealing time.
    function verifyCapture(
        bytes32 root,
        bytes32 recordHash,
```

```
        bytes32[] calldata proof,
        address manufacturer,
        bytes32 lotRoot,
        bytes32 deviceKeyHash,
        bytes32[] calldata deviceProof
    ) external view returns (bool included, uint64 timestamp, uint64 blockNumber, bool deviceValid) {
        Seal memory seal = seals[root];
        timestamp = seal.time;
        if (timestamp == 0) return (false, 0, 0, false);
        blockNumber = seal.blockNumber;
        included = _verify(root, recordHash, proof);
        deviceValid = deviceValidAt(manufacturer, lotRoot, deviceKeyHash, deviceProof, timestamp);
    }

    // @listing-end
    //
    // ---------------------------------------------------------------- internal

    function _isActiveManufacturer(address m) internal view returns (bool) {
        return manufacturerSince[m] != 0 && manufacturerRevokedAt[m] == 0;
    }

    function _verify(bytes32 root, bytes32 item, bytes32[] calldata proof) internal pure returns (bool) {
        bytes32 node = leafHash(item);
        for (uint256 i = 0; i < proof.length; i++) {
            bytes32 sib = proof[i];
            node = node <= sib
                ? sha256(abi.encodePacked(bytes1(0x01), node, sib))
                : sha256(abi.encodePacked(bytes1(0x01), sib, node));
        }
        return node == root;
    }
}
```

A capture is *notarized* once `register` mines. A verifier later calls `verifyCapture` (a view call, no gas) with the batch root, the record hash and its path, and the sensor's manufacturer, lot root, key hash and path; it learns whether the record is sealed, at what time and in which block, and whether that sensor was certified and unrevoked at that time.

I. Verification

A verifier holds a questioned file (bytes E', pixels I'), the public ledger, the record store, and the carrier key k. It needs no claimed payload and no manifest, and it runs Algorithm 2. Both checks end in the same record test: a record is *valid* if its signature verifies under its pk; its hash is included under a sealed root at block time t_b; the sensor was certified by μ in Λ and unrevoked at t_b (`verifyCapture`); and its anchor block precedes the sealing block and carries that block's canonical hash.

(V1) Hard binding. Look up $\mathcal{H}(E')$ among valid records' $\mathcal{H}(E(I))$ or $\mathcal{H}(I)$. A match shows that this exact file existed at t_b (Prop. 1) and was committed in a record signed by a sensor certified at that time (Prop. 4). It is conclusive but brittle: any re-encoding breaks it.

(V2) In-pixel binding. Regenerate the carrier layout from k and the frame's block count N_B; after a block-grid *resynchronization* search (§VII-B), project each carrier block onto $\Phi_{u,v}$ (Eq. 4), de-spread and combine (Eq. 5), and RS-decode $\hat{\mathbf{m}}$. Fetch the records whose payload is $\hat{\mathbf{m}}$, and accept if one of them is valid and the questioned content matches its fingerprint, $d(F(r), F(I'_{\delta^\star})) \le \tau$ (Eq. 8), computed on the image re-aligned by the resync offset $\delta^\star$. The fingerprint test is what makes the payload non-transferable: a payload copied onto other content resolves to a record describing different content.

Media is accepted at the strongest level the evidence supports: CAMERA-ORIGINAL (the exact file a certified sensor committed), TRANSCODED-CONSISTENT (derived from a notarized capture: it carries a payload whose sensor-signed record describes the same scene; the verifier returns the record so the original can be consulted), or the safe UNVERIFIED.

Algorithm 2 PIXELROOT verification of a questioned file (E', I')

Require: ledger, record store, carrier key k, threshold τ

if $\exists$ valid r with $\mathcal{H}(E') \in \{\mathcal{H}(E(I)), \mathcal{H}(I)\}$ **return** CAMERA-ORIGINAL
$(\{B_j\}, \{c_j\}) \leftarrow \mathrm{Select}(\text{SHA-256}(k), N_B, n_c, R)$
$\delta^\star \leftarrow \arg\max_\delta \sum_i |D_i(\delta)|$ {block-grid resync, §VII-B}
for all coded bit i **do**
  $D_i \leftarrow \sum_j c_j \, \langle I'_{B_j+\delta^\star}, \Phi_{u,v} \rangle / \|\Phi_{u,v}\|^2$
  $\hat{b}_i \leftarrow \mathbf{1}[D_i > 0]$; $w_i \leftarrow |D_i|$ {soft conf.}
end for
$\hat{\mathbf{m}} \leftarrow \text{ECC-Decode}(\hat{\mathbf{b}}, \mathbf{w})$
for all $r \in \text{store.lookup}(\hat{\mathbf{m}})$ **do**
  if r valid **and** $d(F(r), F(I'_{\delta^\star})) \le \tau$ **then**
    return TRANSCODED-CONSISTENT, r
  end if
end for
return UNVERIFIED {fail-safe}

VI. SECURITY ANALYSIS

We first state the formal guarantees, then walk through the threat catalogue of §IV.

A. Formal guarantees

Definition 1 (Forgery)**.** *A forgery is a file that the verifier (Alg. 2) accepts as* CAMERA-ORIGINAL *or* TRANSCODED-CONSISTENT*, yet whose content was not produced by a certified* PIXELROOT *sensor executing Alg. 1, up to the transformations the verifier is designed to tolerate.*

Proposition 1 (Hard-binding soundness)**.** *Under a collision-resistant hash $\mathcal{H}$* (A4) *and an immutable ledger* (A3)*, passing V1 implies that the exact file $E^\star$ was committed in a record sealed at block time t_b, i.e. it existed at or before t_b. Producing a distinct $E^\star \neq E$ that passes V1 against a record sealed for E requires a second preimage of $\mathcal{H}$ (generic work $\approx 2^{256}$ for SHA-256); an adversary who also controls the original capture needs only a collision (generic work $\approx 2^{128}$).*

Proof sketch. The leaf is $\mathcal{H}(\texttt{0x00} \| \mathcal{H}(r \| \psi))$ and r contains $\mathcal{H}(E)$; `verifyCapture` recomputes the root from the leaf and its path and compares it with the sealed value. A different $E^\star$ with the same digest is a second preimage (or a collision);

a different leaf with the same root is a collision of the domain-separated node hash; rewriting the sealed root violates A3. □

Proposition 2 (Wrong-key read)**.** *Let the carrier layout be generated by a pseudorandom function keyed with k. To a party without k, the assignment of the N_B blocks to the n_c coded bits and their chip signs is computationally indistinguishable from uniform, so the bits it de-spreads from a marked image are independent of the payload; its probability of decoding the payload is then bounded by Prop. 3. Empirically, reading each of the* 24 *marked Kodak images with* 50 *wrong keys (*1200 *trials) gives mean BER* 0.4996 *and* $0/1200$ *correct recoveries.*

Remark. Prop. 2 bounds *reading* without the key; it does not bound *estimating* the mark. The detector statistic uses only the public basis $\Phi_{u,v}$, so an adversary can estimate the embedded signal from a single marked image and copy or subtract it without knowing k (§IX-D). PIXELROOT's authenticity therefore rests on Prop. 4, not on the secrecy of the layout. (The released prototype seeds its layout generator with 32 bits; the proposition assumes the full-width keyed PRF a deployment uses.)

Proposition 3 (Soft-binding false-accept)**.** *Model the read-back of* (4) *on content that was* not PIXELROOT*-marked as independent bit guesses with per-bit error $p \approx \frac{1}{2}$, so a byte (symbol) matches a given codeword by chance with probability $q = (1-p)^8 = 2^{-8}$. With a Reed–Solomon code of n_{RS} symbols correcting t_{RS} symbol errors, the decoder outputs a given payload with probability at most*

$$\epsilon \;=\; \sum_{j=0}^{t_{\mathrm{RS}}} \binom{n_{\mathrm{RS}}}{j} (1-q)^{\,j}\, q^{\,n_{\mathrm{RS}}-j}, \tag{9}$$

the probability that at least $n_{\mathrm{RS}} - t_{\mathrm{RS}}$ bytes match by chance; it outputs any *of S payloads in the record store with probability at most $S\epsilon$. For RS$(48, 16)$ ($n_{\mathrm{RS}} = 48$, $t_{\mathrm{RS}} = 16$), $\epsilon \le 1.8 \times 10^{-65}$, so even $S = 10^{15}$ stored records leave $S\epsilon < 10^{-49}$.*

Proof sketch. The number of chance byte matches is $\mathrm{Binom}(n_{\mathrm{RS}}, q)$, and the decoder returns a payload only if at least $n_{\mathrm{RS}} - t_{\mathrm{RS}}$ match, giving (9); a union bound over the stored payloads gives $S\epsilon$. Real host content makes bits correlated rather than independent; the empirical counterpart is the $0/24$ false decodes on unmarked images (§IX-C). □

Proposition 4 (Identity non-transferability)**.** *Assume an EUF-CMA-secure signature scheme* (A4)*, an immutable ledger* (A3)*, and on-die sensor keys* (A5)*. If the verifier returns* TRANSCODED-CONSISTENT *for a file $I^\star$, then a record r exists that (i) was signed by a sensor certified on the ledger and unrevoked when r was sealed, and (ii) commits a fingerprint with $d(F(r), F(I^\star)) \le \tau$. An adversary holding no unrevoked sensor key therefore obtains this verdict only by forging a signature (negligible probability) or by presenting content whose fingerprint is within τ of a genuinely captured frame—that is, a derivative of that capture at the fingerprint's resolution.*

Proof sketch. Alg. 2 returns TRANSCODED-CONSISTENT only for a valid record, whose signature verifies under a key whose hash `verifyCapture` places in a certified lot and finds unrevoked at the sealing time. Without that key, a valid (r, ψ) for a new r is an existential forgery. Reusing a genuine (r, ψ) requires passing the fingerprint test against $F(r)$, which r fixes and the signature protects. □

Prop. 4 makes an identity non-transferable *across scenes*; it does not make a capture immune to local edits. Content within τ of a genuine frame includes every benign transcoding, and it also includes edits that keep each fingerprint cell's mean, which an adversary holding the original can construct (§IX-D). This is why a transcoded verdict links to the original capture rather than certifying pixels (§V-G, §VII-D).

Proposition 5 (Capture-time bracket)**.** *If a record with anchor $\beta = (b_\beta, \mathrm{hash}_\beta)$ is accepted, was sealed in block b_s, and hash_β equals the canonical hash of block b_β, then the sensor signed the record after block b_β was produced and no later than block b_s: its capture time lies in $(\mathrm{time}(b_\beta), \mathrm{time}(b_s)]$.*

Proof sketch. The upper bound is Prop. 1. For the lower bound, hash_β is unpredictable before block b_β exists (A3, A4), so a record that contains it, signed by an on-die key (A5), cannot have been signed earlier. The contract stores each batch's sealing block number, and the verifier checks hash_β against the chain through any node (the EVM exposes only the last 256 block hashes on-chain). □

Take-away. Each accepted verdict rests on exponential barriers: the ledger and hash (Prop. 1) for the exact file; the sensor's signature, its on-ledger certification, and the content binding (Prop. 4) for derivatives; and the RS code (Prop. 3) against chance decodes. We now map each catalogued attack to these guarantees.

B. Threat catalogue

Deepfake / synthetic substitution. A fully generated image was never exposed on a certified sensor, so no valid record commits its file or its content. V1 fails, and any payload it carries (decoded by chance, copied, or forged) resolves to no record or to a record whose fingerprint does not match (Prop. 4).

Watermark copy and transplant. An adversary estimates the in-pixel mark from a published image—without the carrier key, from a recompressed copy—and pastes it onto other content [39]. At the watermark level this succeeds (78% of keyless attempts decode the source payload, §IX-D); end to end it fails, because the payload resolves to the source's record, whose fingerprint describes different content ($0/156$ accepted).

Metadata forgery / stripping. Editing or removing metadata cannot create a valid record. Stripping downgrades a credential-only scheme to "unverifiable," but PIXELROOT reads $\mathbf{m}$ from the pixels and resolves the record from the ledger-anchored store (goal G3).

Re-capture (photo-of-a-screen). Re-photographing a screen with a certified sensor produces a *new*, valid record for the re-capture event, with its own time, location and

fingerprint; it does not inherit the original's ledger entry, and back-dating it is prevented (Props. 1, 5). What PIXELROOT attests is that a certified sensor recorded this light at this time, not that the scene in front of it was real. Capture-time liveness signals—depth or time-of-flight maps, focus distance—can be committed as additional record fields so verifiers can flag flat, screen-like scenes; we leave their evaluation to future work.

Replay, back-dating, and pre-staging. The sealing block bounds capture time from above (an artifact cannot be claimed to predate it), and the anchor block bounds it from below (footage prepared earlier cannot be passed off as captured during a later event) (Prop. 5).

PRNU fingerprint-copy. Transplanting a victim's passive sensor noise pattern [30] gains nothing: PIXELROOT's identity is the sensor's signature over a content-bound record, not its PRNU, and a copied noise pattern carries no signature.

Splice, inpaint, and local edits. A splice leaves most carriers intact, so the payload usually still decodes; the fingerprint rejects it when the foreign region changes a tile's coarse content ($24/24$ splices covering 6–12% of the frame rejected, §IX-D), and because no tile is skipped, shifting the image cannot hide it. Smaller patches sometimes stay within τ, and an adversary holding the original can force any local edit to match the fingerprint's cell means. Such images are verified only as derivatives of the capture: the verifier returns the record and the original for comparison, and for forensic media the per-pixel localization layer (§V-B, §IX-C) localizes edits. Pixel-exact integrity is V1's role.

Sensor-key extraction. An invasive attack on a device the adversary owns yields that one sensor's key, with which it can sign records for arbitrary content until the key is revoked; records sealed before the revocation remain valid. The damage is confined to one sensor's identity, but within the window between extraction and revocation it is not bounded in number. Two signals help detect it: the capture counter c (a cloned key that also keeps signing on the original device produces duplicate or out-of-order counters among published records) and implausible capture rates or locations for one key. Shortening the window, e.g. with short-lived sensor sub-keys renewed online, trades against offline operation.

Rogue or compromised manufacturer. Governance revokes the manufacturer; its sensors fail for captures sealed afterwards. A lot certified after a capture cannot validate it retroactively, and a hostile manufacturer cannot revoke another manufacturer's sensors (both enforced by the contract and tested).

Compression-laundering and regeneration. Aggressive re-encoding, mark subtraction (which needs no key, §IX-D), or regeneration with a diffusion model [41] can remove the in-pixel payload. The result is UNVERIFIED—a negative result, never a false positive: PIXELROOT declines to certify; it does not certify a manipulated artifact.

VII. ROBUSTNESS UNDER COMPRESSION

The central engineering tension is that the hard binding (V1) is *fragile* by design—ideal for a pristine original—while real-world distribution demands *semi-fragile* survivability (V2). We therefore analyze the in-pixel channel under lossy coding.

TABLE II
THREAT CATALOGUE MAPPED TO DEFENSES AND GUARANTEES.

Attack	Why it fails	Assump.	Guar.
Deepfake / synth.	no valid record; fingerprint mismatch	A3–A5	P1,P4
Mark copy / transplant	record's fingerprint $\neq$ new content	A3–A5	P4
Metadata forge/strip	V2 reads $\mathbf{m}$; record resolves	A3,A4	P3,P4
Re-capture (screen)	new record; no back-dating	A3–A5	P1,P5
Replay / pre-stage	bracket $(\mathrm{time}(b_\beta), \mathrm{time}(b_s))$	A3–A5	P1,P5
PRNU copy [30]	identity is a signature, not PRNU	A4,A5	P4
Large splice	tile fingerprint mismatch	A4	P4
Local edit (adaptive)	accepted only as derivative; V1 fails	A3,A4	P1,P4
Sensor-key theft	one identity; revocation stops new records	A3	P4
Rogue manufacturer	governance revocation; no retro-lots	A3	P4
Launder / regenerate	fails *safe* to UNVERIFIED	A4	P3

P1–P5 = Props. 1, 2, 3, 4, 5.

A. JPEG

JPEG's quantization matrix scales with the quality factor Q and most aggressively discards high-frequency DCT energy [16]. This is exactly why the carrier (§V-E) lives in a *low-frequency* DCT band rather than at isolated (high-frequency) pixels: the chosen coefficient $(u, v) = (0, 2)$ sits in the lightly-quantized region of the luminance table and is preserved across a wide Q range, while spread-spectrum de-spreading over R blocks (Eq. 5) suppresses both quantization noise and host content by a further $\sqrt{R}$. Our measurements with the production `libjpeg` codec (§IX) confirm the analysis: after RS correction the 128-bit payload is recovered on 22–$23/24$ Kodak images from $Q = 95$ down to $Q = 20$ and on $21/24$ at the extreme $Q = 10$, and the amplitude sweep in §IX locates the graceful decode-failure cliff, below which PIXELROOT returns the safe UNVERIFIED rather than a false positive. The earlier fragile/semi-fragile tension is thus resolved by construction, not asserted.

B. Geometric edits and synchronization

Block-grid embedding is sensitive to operations that move the grid: cropping, translation, padding, and rescaling. PIXELROOT addresses this with an explicit *resynchronization* stage and is honest about its envelope. (i) **Rescaling**: because the carrier is a low-frequency coefficient, moderate down/up-sampling largely preserves it, though not losslessly: under a bilinear resize round-trip the raw BER rises from 0.016 to 0.026 at $0.75\times$ and 0.042 at $0.5\times$, and RS decoding recovers the payload on $22/24$ and $16/24$ images respectively ($22/24$ for a combined JPEG $Q70 + 0.75\times$ attack; §IX). (ii) **Translation/crop**: the verifier searches a range of block-grid offsets δ and keeps the most confident decode (line 3 of Alg. 2). The search must respect the carrier's shape: $\Phi_{0,2}$ is

constant down each column, so the confidence peak is several pixels wide vertically but only one pixel wide horizontally. PIXELROOT therefore searches every horizontal offset on a vertical stride of 4, then refines vertically around the best ($15 \times 57 + 7 = 862$ decodes for ± 28 px). (An earlier search on a stride of 4 in both axes could step over the horizontal peak; on 2% crops it decoded only $20/24$ images at raw BER 0.11.) With the separable search, top-left border crops of 1%, 2% and 3% (shifts of up to 23 px) are recovered on $23/24$ images each at raw BER ≤ 0.020; larger crops were not tested. These crops, like the resize attacks, return an image of the *original dimensions* (content shifted and the border padded, or scaled and scaled back). A crop or rescale that changes the frame's dimensions changes the number of blocks N_B and therefore the carrier layout, and is not recovered by the offset search; supporting it requires a synchronization template that restores the original grid and size. A known synchronization template (a fixed low-frequency pattern) can extend this to scale/anchor estimation, as in the watermarking-sync literature. (iii) **Limits**: large arbitrary crops, rotation, and projective warps that are not covered by the offset/scale search are *not* recovered; critically, PIXELROOT then *fails safe*—the de-spread correlation collapses to noise and the verifier returns UNVERIFIED rather than a false accept (we observe BER ≈ 0.5 when sync fails). Heavy geometric editing therefore degrades to "cannot verify," never to "falsely verified," and the hard binding (V1) plus perceptual-hash "same-scene" layers remain available.

C. Video: MPEG / H.264 / HEVC

Video adds motion-compensated prediction, integer transforms, in-loop deblocking, and GOP structure [17], [18]. The video watermarking literature shows that embedding survives best in mid-frequency transform coefficients and that I-frame perturbations propagate ("drift") through a GOP, so embedding is often confined to portions of the GOP [44]–[46]. For PIXELROOT, the practical design is: embed the per-frame (or per-keyframe) payload at capture, and commit a Merkle root over keyframe hashes so that *frame-level* verification localizes tampering. Learned, codec-aware embedding has recently been shown to survive non-differentiable H.264 [46] and is a natural upgrade path.

D. The semantic-authenticity boundary

A subtle limitation deserves emphasis. A benign platform filter may alter carrier pixels enough to fail V2 even though the image's *meaning* is unchanged—a false alarm—while a malicious edit might, in principle, preserve carriers. PIXELROOT therefore certifies *consistency with a notarized original*—exactly for V1, and up to the fingerprint tolerance τ for V2—not semantic equivalence. A local edit that keeps every tile within τ of the original (for example, a small object removed from a textured region) passes V2; edits that change a tile's coarse content, such as the 6–12% splices of §IX-D, do not. We argue this is the right primitive: it cleanly separates "bit-for-bit original" (V1), "faithfully transcoded" (V2), and "cannot establish provenance," and it composes with perceptual-hash and content-similarity layers [49] for softer "same-scene" judgments.

VIII. APPLICATIONS: HUMAN-IN-THE-LOOP TRUST

PIXELROOT is designed so that a human or institutional verifier remains in the loop with hardware-rooted evidence rather than a model's opinion.

Legal and evidentiary. Chain-of-custody for photographic/video evidence benefits from an immutable, independently timestamped commitment made at capture, reducing disputes over authenticity and time of creation [28].

Journalism and third-party (3P) verification. Platforms and fact-checkers can verify an asset against the public ledger without contacting the source—valuable for conflict and human-rights documentation, where ProofMode-style field capture has already been deployed [11].

Life-saving / high-assurance. Insurance inspections, telemedicine imagery, infrastructure monitoring, and emergency reporting need affirmative proof that an image is a real, current capture of a real scene—exactly the "prevent rather than detect" posture argued by secure-capture vendors [10].

Platform-scale screening. Social platforms can perform an $O(1)$ ledger lookup at upload to label assets "camera-original," "transcoded-but-consistent," or "unverified," shifting the default from "trust until debunked" to "label by provenance."

IX. EVALUATION AND REFERENCE IMPLEMENTATION

A. Reference implementation

We provide an open software prototype of the PIXELROOT core (payload layout, SHA-256 seeding, PRNG carrier selection, gain-value generation, image hashing, and a register/verify service with simulated notarization). It implements both a legacy v1 (120-bit) and the current v2 (128-bit) payload, the latter fixing a coordinate-packing precision bug. The prototype validates the encode/verify round-trip and the metadata→seed→position determinism end-to-end before silicon. A second, *real-image* harness (Python; `numpy`, Pillow's libjpeg/WebP, `scikit-image`, and a real `reedsolo` code) runs the full evaluation below on the public Kodak set; a claims-to-experiments matrix and reproduction instructions are released in `experiments/EXPERIMENT_PLAN.md` at https://github.com/mihirkavi/PixelRoot. The device-identity layer is implemented as a Solidity registry (`contracts/`, 18 Hardhat tests) and a Python reference of the sensor key, signed records, content fingerprint, and verifier (`ledger.py`, `device_identity.py`, 21 unit tests), with Merkle test vectors shared between the two implementations.

B. Micro-benchmark of the core (measured)

Table III reports the cost of the capture-time operations, measured with the Python reference implementation on a commodity CPU (`e5_crypto.py`, `e6_device_identity.py`) and gas measured on the Hardhat EVM. The on-device cryptographic work is small next to an exposure: generating the carrier layout for a 1080p frame takes 1.7 ms, a SHA-256 commitment over the 1080p

TABLE III
MEASURED COST OF THE CAPTURE AND VERIFICATION PATH (PYTHON REFERENCE ON A COMMODITY CPU; GAS ON THE HARDHAT EVM, `solc` 0.8.24).

Operation	Cost
Carrier layout, 1080p (384 bits, $R=84$)	1.7 ms
SHA-256 over 1080p / 6 MP RGB frame	4.2 / 12.5 ms
Record: fingerprint + ECDSA P-256 sign	9.0 ms
ECDSA P-256 verify	0.15 ms
Verify exact camera file end to end	4.2 ms
Merkle root over 10^4 records	17.3 ms
Inclusion proof, 10^4 batch	14×32 B
`register` (any batch size)	47,016 gas
per image at $N=10^4$	4.70 gas
`registerLot` / `revokeDevice`	51,626 / 65,859 gas
`verifyCapture`	view call (free)

frame 4.2 ms (12.5 ms for a 6 MP frame), and forming and signing a capture record 9.0 ms, all in unoptimized Python; a sensor's hardware hash and signature engines are faster.

C. Measured robustness of the in-pixel channel

We evaluate the spread-spectrum embedding (§V-E) on *real public images* with *real image codecs*, following the current watermark-robustness benchmarking practice of WAVES [68] (a broad attack suite reported as performance-vs-quality with PSNR/SSIM). **Methodology.** The 24 Kodak natural photographs (768×512) [69] carry a 128-bit payload protected by a real Reed–Solomon code $\mathrm{RS}(48, 16)$ (`reedsolo`); the 384 coded bits are spread with repetition over 8×8 blocks at amplitude $A = 4$ gray levels on the $(0, 2)$ carrier, embedded in the luma channel. Attacks use *actual libjpeg / WebP* (with 4:2:0 chroma subsampling), Gaussian noise and blur, bilinear resize round-trips, photometric (brightness/contrast/gamma) edits, translational crop with grid resynchronization, and rotation/ resized-crop. Decoding uses the host-suppressed correlation de-spread of Eq. (5) followed by RS decoding. The harness is released (§IX, `experiments/`).

Fidelity. Embedding is imperceptible: mean PSNR 38.2 dB and SSIM 0.942 between original and marked frames (the per-frame fidelity rises to $\sim$41 dB at $A=3$; Table V). **Robustness.** Table IV reports the raw (pre-ECC) coded-bit BER and the *end-to-end* payload-decode success after RS correction. On real content the raw channel is not error-free—mean BER $\approx 1.6\%$, content-dependent up to 13% on the most textured frame—but the RS code recovers the exact 128-bit payload on 22–23/24 images through real JPEG from $Q = 95$ down to $Q = 20$ (and 21/24 at the extreme $Q = 10$), Gaussian noise to $\sigma = 20$, light blur ($\sigma = 0.5$), and translational crops of 1–3% with resync, and on 21–22/24 under WebP, resize to 0.75–1.5×, photometric edits, and combined JPEG+resize. Recovery is weaker under stronger blur ($\sigma=1.0$: 19/24) and 0.5× resize (16/24). Failures are honest and informative: a single very-high-frequency frame exceeds the ECC capacity at $A \leq 6$, and *geometric* rotation ($\pm$1–2°) and resized-crop drive BER to ≈ 0.5—a documented limitation requiring explicit rotation/scale synchronization (§X). **Operating curve.**

TABLE IV
MEASURED IN-PIXEL-CHANNEL ROBUSTNESS ON **24 REAL KODAK IMAGES** WITH **REAL LIBJPEG/WEBP** CODECS (128-B PAYLOAD, $\mathrm{RS}(48, 16)$, $A=4$; PSNR 38.2 DB, SSIM 0.942). "RAW BER" IS THE PRE-ECC CODED-BIT ERROR; "DECODE" IS EXACT POST-ECC PAYLOAD RECOVERY.

Attack	raw BER	decode
JPEG $Q95$–$Q20$	0.016	22–23/24
JPEG $Q10$	0.022	21/24
WebP $q80$, $q60$	0.021–0.024	21–22/24
Gaussian noise $\sigma \leq 20$	0.017	23/24
Gaussian blur $\sigma = 0.5$	0.018	23/24
Gaussian blur $\sigma = 1.0$	0.033	19/24
Resize 0.75×, 1.5×	0.025	22/24
Resize 0.5×	0.042	16/24
Bright/contrast/gamma $\pm$10%	0.017	21–23/24
Crop 1–3% + resync	0.019	23/24
JPEG$Q70$ + resize 0.75×	0.026	22/24
JPEG $Q5$ (beyond range)	0.221	0/24
Rotation $\pm$1–2° *[geom. limit]*	0.50	0/24
Resized-crop *[geom. limit]*	0.49	0/24
Wrong-key read (1200 trials)	0.4996	0/1200
Forgery / unmarked (24 trials)	0.494	0/24
Copy attack → source payload	—	15/17

TABLE V
AMPLITUDE OPERATING CURVE AT JPEG $Q40$ (ALL 24 KODAK IMAGES): IMPERCEPTIBILITY VS. POST-ECC ROBUSTNESS, SHOWING THE GRACEFUL FAILURE CLIFF.

A (gray)	PSNR (dB)	raw BER	decode
0.5	$\infty^\dagger$	0.504	0/24
1.0	51.2	0.270	0/24
1.5	48.2	0.179	2/24
2.0	44.2	0.090	8/24
3.0	41.2	0.043	15/24
4.0	38.2	0.016	23/24
6.0	35.2	0.005	23/24

†At $A=0.5$ rounding to integers leaves most pixels unchanged.

Table V and Fig. 7(b) sweep amplitude at a fixed strong attack (JPEG $Q40$) over all 24 images, exposing a genuine cliff: post-ECC decode rises from 0/24 at $A = 0.5$ to 23/24 at $A = 4$ (38 dB), so $A = 4$ is the imperceptible-yet-robust operating point. **Security / false-accept.** Reading a marked image with 50 wrong keys across all images (1200 trials) gives mean BER 0.4996 and 0/1200 recoveries; running the correct-key decoder against the 24 *unmarked* images (a deepfake stand-in) against a claimed payload gives mean BER 0.494 and 0/24 false accepts. These match the $p \approx \frac{1}{2}$ model of Props. 2–3, whose closed-form RS false-accept probability is $\leq 1.8 \times 10^{-65}$ for this code. A *copy* attack, however, succeeds at the watermark level: lifting the marked residual from image A onto a different image B makes B decode to A's payload in 15/17 cases. The in-pixel channel alone does not make an identity non-transferable; §IX-D shows that the signed, content-bound record does.

Optional per-pixel localization layer. When the sensor supports per-pixel gain (§V-B), PIXELROOT adds a second, keyed

TABLE VI
PER-PIXEL TAMPER-LOCALIZATION ON **24 REAL KODAK IMAGES** WITH **48 REAL SPLICES** (RECT/ELLIPSE, PIXEL-LEVEL GROUND TRUTH). METRICS FOLLOW THE FORGERY-LOCALIZATION LITERATURE; LOCALIZATION GRANULARITY IS 32 PX.

Metric	Pristine	JPEG $Q90$
ROC-AUC (block)	0.974	0.954
Pixel-F1 / p-F1	0.74	0.56
Pixel-IoU	0.59	0.39
Block TPR / FPR	0.914/0.037	0.927/0.104

high-resolution pattern that serves as a spatial integrity map. On the 24 real Kodak images we embed a keyed $\pm\delta$ ($\delta = 4$ gray levels) per-pixel pattern, then create 48 *real splices* (rectangular and elliptical foreign regions copied from other photographs, with pixel-level ground-truth masks), and detect tampering with a per-block matched-filter statistic (block 32 px). We report the metrics standard in the forgery-localization literature (pixel-F1, permutation p-F1, IoU, ROC-AUC) [70], [71]. On pristine media the layer reaches ROC-AUC 0.974, pixel-F1 0.74, IoU 0.59, and a block true/false-positive rate of 0.914/0.037 (Table VI, Fig. 7(c)). Under JPEG $Q90$ it degrades gracefully to AUC 0.954 rather than collapsing into false alarms. This layer carries no identity bits—it complements, and does not replace, the robust payload channel—and is intended for lightly-processed forensic media.

PRNU baseline (passive comparison). As a reference point we implemented the standard passive sensor-fingerprint pipeline (denoising-residual fingerprint, normalized cross-correlation, and PCE [14], [15]) on the same real content. Camera attribution is accurate on clean images and, in this two-camera setting with a simulated multiplicative fingerprint, stays accurate down to JPEG $Q50$; but the genuine–impostor NCC margin narrows from $+0.019$ (clean) to $+0.004$ at $Q50$ and PCE drops by $\sim 18\%$ (Fig. 7(d)). A passive fingerprint also only says *which camera*, needs reference images of that camera, and can be copied; PIXELROOT's active, error-corrected payload instead carries an identifier that resolves to a signed record (it is recovered on 21/24 images at $Q10$, Table IV). A realistic multi-camera PRNU comparison on RAW data is future work (F1).

D. Device identity and content binding (E6)

We evaluate the full verification path of §V-G on the same 24 Kodak images. **Setup.** One manufacturer admitted to the registry certifies a lot of eight sensors, each holding an on-die ECDSA P-256 key; every image is marked as in §IX-C ($A = 4$, RS$(48, 16)$), encoded by the "camera" as JPEG $Q95$, and described by a sensor-signed record (Eq. 7); the record hashes are sealed as one Merkle batch. The ledger is an executable model of the contract of §V-H; the contract itself passes an 18-case test suite on the Hardhat EVM covering the same state machine, Merkle compatibility with the Python implementation through shared test vectors, the leaf/node domain separation, the per-manufacturer lot keys (including a front-running race), the sealing block number, and every revocation rule. To avoid tuning on the test set, the fingerprint threshold is calibrated on the 12 even-indexed images (midpoint between the largest genuine distance under the benign suite of Table IV and the smallest distance of 25–35% splices), and *every result below is on the* 12 *held-out images*, with Wilson 95% intervals where rates are reported.

Fingerprint margin. Calibration gives $\tau = 0.482$. On held-out images the largest genuine distance under any benign attack is 0.197, while different images are at least 1.085 apart (median 2.55).

Genuine media (Table VII). The camera's own JPEG verifies as CAMERA-ORIGINAL on 12/12 images through V1 alone, with no dependence on the watermark. Processed copies verify as TRANSCODED-CONSISTENT in 358 of 396 cases (90%, CI 0.87–0.93): 11/12 images across JPEG down to $Q20$, noise, blur, resize 0.75–1.5×, photometric edits, 1–3% crops and combined attacks, and 9–10/12 under JPEG $Q10$, WebP $q60$, brightness ×1.1 and 0.5× resize. All 38 UNVERIFIED genuine verdicts are payload-decode failures (mostly the texture-heavy frame of §IX-C); the fingerprint rejected none. None of the 12 unmarked originals is accepted.

Cross-scene forgeries (Table VIII). The keyless copy attack estimates the mark from a JPEG $Q75$ copy of the source by the detector's own public projection and high-pass and adds 1.5× the estimate to another image; the forgery decodes the source's payload in 61/78 attempts (78%), and 70/78 with the original in hand. The content-bound record rejects all 156 (CI upper bound 0.05 per attack), and all 24 splices covering 6–12% of the frame, including the 21 whose payload still decodes.

Local edits within one capture. Smaller pasted patches are sometimes within τ: 3/12 at 64×64 px (about 1% of the frame), 1/12 at 96×96, 0/12 at 128×128, and 2/12 for a 128-px patch near the border followed by an 8-px shift (a whole foreign tile row followed by the same shift is rejected 12/12, so a crop no longer hides content). An *adaptive* adversary who holds the original does better: after pasting a 96×96 foreign patch it re-imposes the original's mean on every fingerprint cell, and 11/12 such images are accepted. The pasted detail survives only below the cell size (about 5 px here): its correlation with the donor is 0.52, and it changes the patch by 17.8 gray levels on average—visible on inspection (Fig. 8). These results are why PIXELROOT defines a transcoded verdict as *derivation from a notarized capture*, returns the record (and, when published, the original file) for comparison, and reserves pixel-exact certification for V1 (§V-G).

Ledger and record attacks. All eleven record- and ledger-level cases in Table VIII behave as specified, including time-scoped revocation in both directions, an anchor block later than the seal, an anchor hash that is not the chain's, and a burst of two captures sharing one payload (both resolve).

Cost. Signing a capture (fingerprint and ECDSA) takes 9.0 ms in Python on a commodity CPU; ECDSA verification takes 0.15 ms, and verifying an exact camera file end to end 4.2 ms. A record is ≈ 18 KB as hex-encoded JSON (about half that in binary), off-chain. On-chain (Hardhat EVM, `solc` 0.8.24), sealing a batch costs 47,016 gas regardless of its size—4.70 gas per image at $N = 10^4$—certifying a lot costs 51,626 gas, revoking a sensor 65,859 gas, and verification is

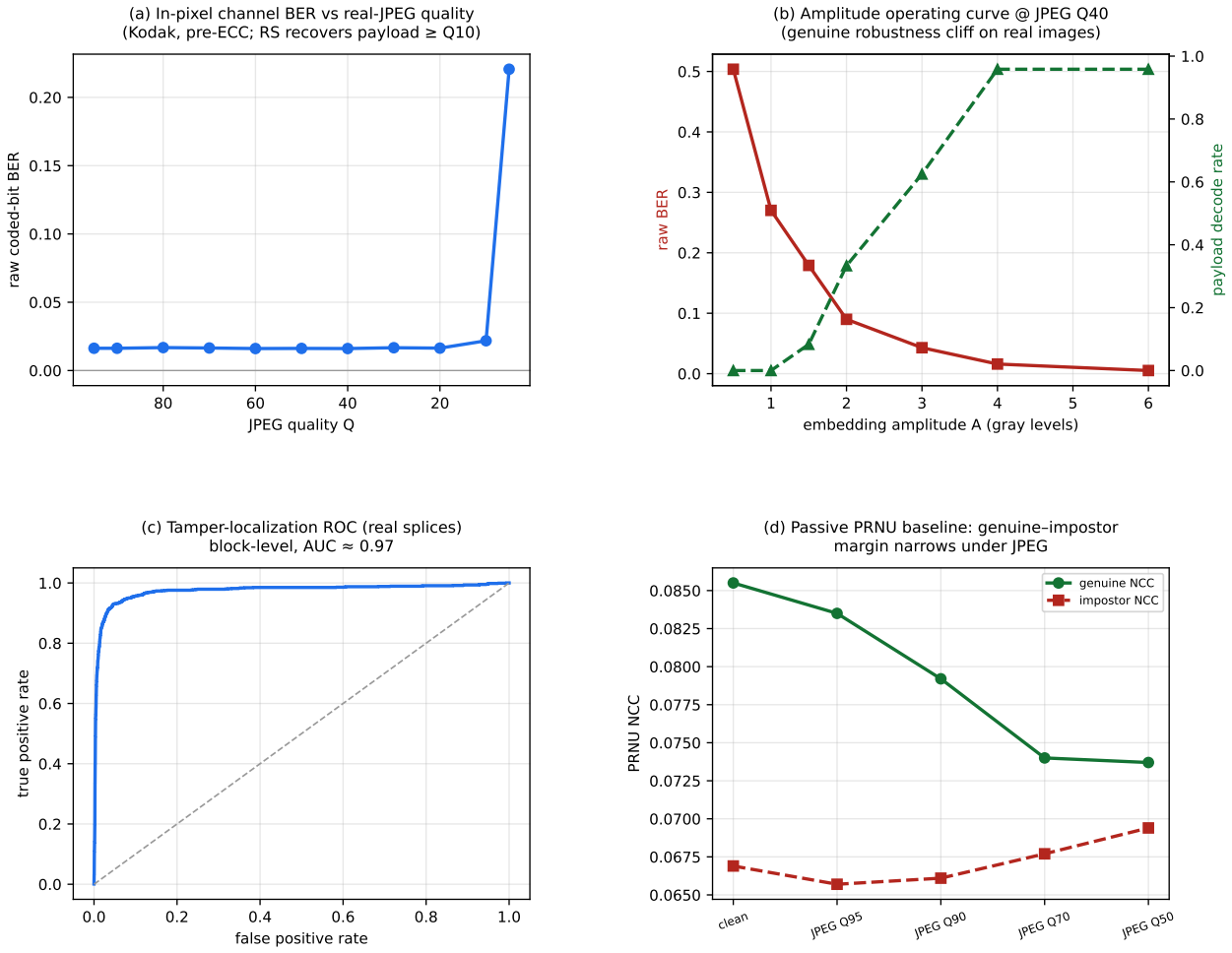


Fig. 7. Measured results on real public images. (a) In-pixel channel raw BER vs. real-JPEG quality on Kodak (nearly flat through $Q10$, cliff at $Q5$; RS recovers the payload on 21–23/24 images through $Q10$). (b) Amplitude operating curve at JPEG $Q40$ over all 24 images: a genuine robustness cliff. (c) Tamper-localization ROC on real splices (AUC $\approx$0.97). (d) Passive PRNU baseline: the genuine–impostor correlation margin narrows under JPEG.

TABLE VII
E6 GENUINE-MEDIA VERDICTS ON THE **12 HELD-OUT KODAK IMAGES** (CO = CAMERA-ORIGINAL, TC = TRANSCODED-CONSISTENT, U = UNVERIFIED); τ CALIBRATED ON THE OTHER 12. EVERY U IS A PAYLOAD-DECODE FAILURE.

Condition	**CO**	**TC**	**U**
Camera JPEG (exact file)	12	0	0
JPEG $Q95$–$Q20$	0	11	1
JPEG $Q10$; WebP $q60$	0	10	2
WebP $q80$; noise $\sigma \leq 20$; blur $\sigma \leq 1$	0	11	1
Resize 0.75×, 1.5×	0	11	1
Resize 0.5×	0	9	3
Bright./contrast/gamma ±10%	0	10–11	1–2
Crop 1–3% + resync	0	11	1
JPEG+resize; JPEG+noise	0	11	1
All processed copies	0	358	38
Unmarked originals	0	0	12



Fig. 8. Adaptive local edit accepted as TRANSCODED-CONSISTENT (E6). Left: marked original. Centre: a 96×96 patch from another photograph, pasted and then forced to the original's fingerprint-cell means, after JPEG $Q90$. Right: the donor. The edit is visible on inspection and appears in a comparison with the original that the record links to; only V1 certifies exact pixels.

a free view call.

Scope and threats to validity. These results validate the *algorithmic* core—carrier design, host-suppressed spread-spectrum detection, robustness to *real* JPEG/WebP/noise/blur/resize/photometric/crop attacks, false-accept behavior, and tamper localization—on *real public imagery* with *real codecs* and a *real Reed–Solomon* code, independent of silicon. They embed an *additive* luma perturbation, whereas a sensor gain change is *multiplicative*: the imprinted signal scales with scene brightness, weakening in dark regions and clipping at saturation. They do not yet model this, demosaicing from the Bayer/RAW domain,

TABLE VIII
E6 ATTACKS ON THE 12 HELD-OUT IMAGES: WATERMARK-LEVEL SUCCESS VERSUS ACCEPTANCE AS TRANSCODED-CONSISTENT BY ALG. 2.

Attack	Payload decodes	Accepted
Cross-scene		
Keyless copy (from JPEG $Q75$)	$61/78$	$0/78$
Copy with original residual	$70/78$	$0/78$
Splice 6–12% of frame, $Q90$	$21/24$	$0/24$
Local edits within one capture		
Patch 64 px, $Q90$	$11/12$	$3/12$
Patch 96 px, $Q90$	$11/12$	$1/12$
Patch 128 px, $Q90$	$11/12$	$0/12$
Foreign tile row + 8-px shift	$8/12$	$0/12$
128-px patch near border + shift	$11/12$	$2/12$
Adaptive: 96-px patch + cell means	$11/12$	$11/12$
Record and ledger		
Record edited (fingerprint, file hash)	—	$0/2$
Uncertified sensor key; borrowed signature	—	$0/2$
Revoked sensor, sealed after revocation	—	$0/1$
Revoked sensor, sealed before revocation	—	$1/1$ (valid)
Anchor after seal; forged anchor hash	—	$0/2$
Record not notarized	—	$0/1$
Burst: two records, one payload	—	$2/2$ (valid)

real per-column gain calibration error, lens/optical effects, or the analog photo-electron domain. The device-identity results (E6) use a software model of the sensor key and an executable model of the ledger; the contract is tested on a local EVM, not on a public network; the PRNU comparison uses real Kodak *content* with the textbook multiplicative sensor model (per-camera RAW datasets are access-gated). The full harness, datasets script, and figures are released for reproduction.

E. Remaining silicon- and codec-dependent validation

The following require hardware or a full pipeline and are future work: (F1) RAW/Bayer-domain capture and a multi-camera PRNU study on Dresden/VISION; (F2) video robustness vs. H.264/HEVC QP and GOP placement with keyframe-level Merkle commitments; (F3) an end-to-end re-capture and PRNU-copy [30] study, including capture-time liveness fields; (F4) shutter-to-inclusion latency and finality on a public network (gas is measured, §IX-D); (F5) a silicon study of per-region/per-pixel gain modulation on a sensor evaluation board, including multiplicative gain on RAW data through a full ISP; and (F6) on-die key generation and signing in a sensor secure element.

X. DISCUSSION: LIMITATIONS, PRIVACY, STANDARDIZATION

Hardware dependency and assurance tiers. PIXELROOT's strongest guarantees require per-column/per-region gain and secure-element support (§V-A). We make the trust gradient explicit rather than hiding it, reporting one of three tiers with every verdict: *Tier A (sensor-rooted)*—modulation imposed in the photo-electron domain by gain hardware; *Tier B (ISP-attested)*—the identical modulation applied in an attested ISP on legacy devices, trustworthy only up to the attestation; and *Tier C (software/notarize-only)*—a C2PA/ProofMode-style capture-time hash with no in-pixel binding. Verifiers and policies must not conflate tiers.

Verification trust model. Every check in Alg. 2 uses public information. The hard binding (V1) needs only the file, the record and the ledger. The in-pixel binding (V2) additionally needs the carrier key k to locate the carriers; because authenticity rests on the sensor's signature and the content binding (Prop. 4) rather than on the secrecy of k, an OEM can publish its carrier key and let any platform, court, or viewer run V2. An OEM that prefers to keep k private (to make casual removal of the mark slightly harder) can instead operate a verification service; the security of an accepted verdict is the same either way.

Privacy-preserving notarization. A naive design that wrote sensor ID, time, and precise GPS to a public, immutable ledger would be actively dangerous for the very journalists and activists PIXELROOT aims to protect, and would violate data-protection law. PIXELROOT is therefore designed so that *no raw PII ever touches the chain*. (i) The only on-chain object is an opaque Merkle root of record hashes, alongside lot roots of sensor-key hashes and the revocations; the record, the payload $\mathbf{m}$ and the image live off-chain. (ii) The leaf $h = \text{SHA-256}(r\|\psi)$ covers a randomized ECDSA signature, so the chain reveals nothing about r, and records are published only for images their owner chooses to publish. (iii) Location is optional, omittable, and may be quantized coarsely or replaced by a salted commitment disclosed only under subpoena. (iv) A sensor's public key appears on-chain only inside a lot root (and in clear only if the sensor is revoked); the in-pixel identifier is a rotating pseudonymous handle. A published record does reveal its sensor key, so two published images from one sensor are linkable; replacing ECDSA with a group or anonymous-attestation signature [72] would let a verifier learn only that *some* certified sensor in lot Λ signed, and is the natural privacy upgrade; with it, a holder could also prove properties of a capture ("taken by a certified PIXELROOT sensor before block b_s") without revealing which sensor, which we leave to future work. (v) Because only opaque hashes are on-chain, the off-chain record can be deleted to honor a GDPR/erasure request while the immutable proof-of-existence remains a meaningless digest—squaring immutability with the right to be forgotten. In high-risk settings the capturing party may also defer notarization (commit a sealed hash, disclose context later).

Why a public ledger and not just a timestamping authority? Proposition 1 only needs a trustworthy, append-only timestamp, which an RFC 3161 time-stamping authority (TSA) also provides. We anchor to a public blockchain for properties a single TSA lacks: *no single point of trust or failure* (a compromised or coerced CA can back-date or deny service, and certificate compromise has already forced wholesale revocation in a deployed C2PA camera pipeline [13]); *public, permissionless auditability* by any court, platform, or adversary without trusting the issuer; and *censorship resistance*, important when the capturing party and the authority are adversaries (conflict zones, state actors). The same

ledger also carries sensor certification and revocation, so a verifier answers "was this sensor trusted *when* this capture was sealed?" from one public source, with no dependence on a vendor's revocation service—a question a certificate authority alone answers poorly once revocation lists are ignored or unavailable [13]. The cost—latency and gas—is amortized to $O(1)$ per capture by Merkle batching (§V-H); measured gas is $47{,}016$ per batch (§IX-D). A *hybrid* is natural and recommended: a TSA countersignature for instant, cheap proof plus periodic blockchain anchoring of the TSA's own Merkle roots, as OpenTimestamps does [11]; PIXELROOT treats the ledger as the trust-minimizing anchor and the TSA as a low-latency accelerator. An append-only transparency log in the style of Certificate Transparency [73] offers similar auditability with lower latency but depends on its operators and monitors; it can likewise serve as a fast path whose roots are anchored on the chain.

Failure-safe semantics. As emphasized, PIXELROOT declines rather than falsely certifies under heavy laundering or unsynchronizable geometric edits; downstream policy must treat "unverified" as "unknown," not "fake."

Adoption path and honest efficacy ceiling. PIXELROOT cannot verify media from cameras that never marked it, so its value grows with deployment and is *not* a universal deepfake solution. We therefore propose a phased, backward-compatible rollout: (1) software Tier C in capture apps and the notarization contract today (no hardware change); (2) ISP Tier B via firmware on existing devices; (3) Tier A as OEMs add the modest per-column/per-region gain capability. Throughout, verification is opt-in and additive: unmarked media is labeled "unverified," never penalized as fake, so the system shifts high-stakes workflows (courts, newsrooms, insurance) from "trust until debunked" to "label by provenance" without breaking the long tail of legacy content.

Toward an OEM standard. The historical lesson of JPEG [16] and 3GPP is that interoperability emerges from a shared standard, not point products. We propose PIXELROOT as a cross-OEM capture-provenance profile that (a) fixes payload layout, ECC, the keyed selection function, and the carrier basis; (b) reuses C2PA manifests as the transport for assertions; (c) standardizes the capture-record format, the content fingerprint, and the registry contract interface—lot certification, revocation, sealing and `verifyCapture`—so any platform can verify any vendor's media. Consumer hardware-backed C2PA signing [10], [12] shows the ecosystem is ready for the signing half; PIXELROOT adds the in-pixel, ledger-witnessed half.

XI. Conclusion and Future Work

Reactive deepfake detection cannot win a generator-vs-detector arms race, and metadata-only provenance breaks the moment a credential is stripped. PIXELROOT relocates the root of trust into the pixels and onto a public ledger: an error-corrected payload embedded by the CMOS sensor at capture, a capture record signed by a key that only that sensor holds, and a blockchain that certifies the sensor, seals the record, and records any revocation. We showed that the in-pixel mark alone is not enough—it can be copied onto other content without any key—and that binding it to a sensor-signed content fingerprint makes the identity non-transferable: on held-out images no cross-scene copy or large splice was accepted, while 90% of processed genuine copies verified through compression, resizing, photometric edits and crops. We also showed the limit of any tolerant binding: an adversary who holds the original can make local edits that keep the fingerprint, so a transcoded verdict means *derived from* a notarized capture, which the verifier returns for comparison, and only the camera-original verdict certifies exact pixels. We are careful about what this paper establishes: the algorithmic core and the identity layer are *implemented and measured* on real public images with real codecs; the registry contract is *implemented and tested* on a local EVM with measured gas, but not yet deployed on a public network; and the sensor-level gain hardware and on-die keys are a *feasibility analysis* grounded in shipping process technology, not a silicon result. Future work includes silicon-level validation on sensors with per-region gain control, codec-aware learned embedding for video with per-segment records and Merkle trees of segments [46], capture-time liveness fields against screen re-capture, anonymous-attestation signatures for unlinkable records, and pursuing PIXELROOT as an open cross-OEM standard. We view this as a foundational—if not all-encompassing—step toward driving deepfake-driven disinformation toward practical irrelevance for media that matters.

Acknowledgment

Use of generative AI. In accordance with IEEE and arXiv policy, the author discloses that Claude, a large language model by Anthropic (used through Claude Code), assisted substantially in preparing this manuscript and its software. It drafted and revised text, most extensively in the device-identity design (§V-G–§V-H), the security analysis (§VI), the identity evaluation (§IX-D), and the discussion (§X), and edited the remaining sections for consistency with the measured results; it wrote and ran, under the author's direction, the device-identity and evaluation code, the registry contract and its tests, and the attack experiments; and it checked every reported number against the experiment outputs. The author directed the work, reviewed all AI-assisted text, code, and results, and takes full responsibility for the content of this article.

Conflict of Interest

The author declares that there is no conflict of interest regarding the publication of this article. The work received no external funding, and the author has no financial or personal relationships that could have inappropriately influenced it.

Reproducibility and Disclosure

The reference implementation and the evaluation harness (https://github.com/mihirkavi/PixelRoot)—the keyed spread-spectrum embed/extract of §V-E, the Reed–Solomon error-correction layer, the WAVES-aligned attack suite, and the tamper-localization and PRNU pipelines, the device-identity reference and verifier, and the registry contract with its test suite—are part of the open PIXELROOT project. All measured

numbers in §IX were produced on the public Kodak image set with production codecs (`libjpeg` JPEG and `libwebp` WebP) and are reproducible end-to-end from the released scripts, a `requirements.txt`, and the contract's locked `npm` dependencies (the Solidity compiler is pinned to 0.8.24). The ledger in the Python experiments is an executable model of the contract, and the contract is tested on a local EVM; neither has been deployed to a public network. The sensor keys are software stand-ins for on-die keys, and the silicon-dependent per-pixel gain mechanisms of §V-B are a feasibility analysis grounded in the cited fabrication literature, not fabricated results.

Data Availability

The Kodak TrueColor image suite used in the experiments is publicly available. The provenance payloads are synthetic and contain no personal data. No human or animal subjects were involved in this work.

References

[1] F.-A. Croitoru, A.-I. Hiji, V. Hondru, N.-C. Ristea, P. Irofti, M. Popescu, C. Rota, R. T. Ionescu, F. S. Khan, and M. Shah, "Deepfake media generation and detection in the generative AI era: A survey and outlook," *arXiv preprint arXiv:2411.19537*, 2024.

[2] L. Verdoliva, "Media forensics and DeepFakes: An overview," *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 5, pp. 910–932, 2020.

[3] Resemble AI, "The 2025 deepfake threat report," Resemble AI, Tech. Rep., 2025, https://www.resemble.ai/resources/2025-deepfake-threat-report.

[4] Surfshark Research, "Deepfake-related fraud losses by social media platform, 2025," Surfshark, Tech. Rep., 2025, https://surfshark.com/research/chart/deepfake-social-media-fraud.

[5] IRONSCALES, "Beyond detection: The reality of deepfake attacks (fall 2025 threat report)," IRONSCALES, Tech. Rep., 2025.

[6] N. A. Chandra *et al.*, "Deepfake-Eval-2024: A multi-modal in-the-wild benchmark of deepfakes circulated in 2024," *arXiv preprint arXiv:2503.02857*, 2025.

[7] H. Sun *et al.*, "The tug-of-war between deepfake generation and detection," *arXiv preprint arXiv:2407.06174*, 2024.

[8] A. Kaur, A. N. Hoshyar, V. Saikrishna, S. Firmin, and F. Xia, "Deepfake video detection: Challenges and opportunities," *Artificial Intelligence Review*, vol. 57, no. 6, 2024.

[9] Coalition for Content Provenance and Authenticity (C2PA), "C2PA technical specification (content credentials), v2.x," 2024, https://spec.c2pa.org.

[10] Truepic, "Controlled capture and the Truepic lens SDK: Hardware-backed C2PA media provenance," 2024, https://www.truepic.com.

[11] Starling Lab and Guardian Project, "ProofMode: Authenticated media capture and field verification," 2023, https://starlinglab.org/prototypes/proofmode-authentication/.

[12] Google, "How Pixel and Android are bringing a new level of trust to your images with C2PA content credentials," 2025, https://blog.google/security/pixel-android-trusted-images-c2pa-content-credentials/.

[13] E. Golaszewski, N. Krawetz, A. T. Sherman, E. Zieglar, S. K. Matukumalli, R. Yus, C. L. Kegley, M. Barthel, W. Bowman, B. Barot, and K. Kullman, "Verifying provenance of digital media: Security analysis of C2PA and its implementation," Cryptology ePrint Archive, Paper 2026/804, 2026. [Online]. Available: https://eprint.iacr.org/2026/804

[14] J. Lukáš, J. Fridrich, and M. Goljan, "Digital camera identification from sensor pattern noise," *IEEE Transactions on Information Forensics and Security*, vol. 1, no. 2, pp. 205–214, 2006.

[15] M. Chen, J. Fridrich, M. Goljan, and J. Lukáš, "Determining image origin and integrity using sensor noise," *IEEE Transactions on Information Forensics and Security*, vol. 3, no. 1, pp. 74–90, 2008.

[16] G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992.

[17] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003.

[18] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012.

[19] National Institute of Standards and Technology, "FIPS PUB 180-4: Secure hash standard (SHS)," NIST, Tech. Rep., 2015.

[20] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

[21] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology – CRYPTO '87*, 1987, pp. 369–378.

[22] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008, https://bitcoin.org/bitcoin.pdf.

[23] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," 2014, ethereum Yellow Paper.

[24] J. Benet, "IPFS – content addressed, versioned, P2P file system," *arXiv preprint arXiv:1407.3561*, 2014.

[25] Trusted Computing Group, "TPM 2.0 library specification and device attestation," Trusted Computing Group, Tech. Rep., 2019.

[26] Y. Mirsky and W. Lee, "The creation and detection of deepfakes: A survey," *ACM Computing Surveys*, vol. 54, no. 1, pp. 1–41, 2021.

[27] F.-A. Croitoru *et al.*, "PRNU-Bench: A novel benchmark and model for PRNU-based camera identification," in *arXiv preprint arXiv:2509.17581*, 2025.

[28] P. Korus, "Digital image integrity – a survey of protection and verification techniques," *Digital Signal Processing*, vol. 71, pp. 1–26, 2017.

[29] A. Piva, "An overview on image forensics," *ISRN Signal Processing*, vol. 2013, pp. 1–22, 2013.

[30] M. Goljan, J. Fridrich, and M. Chen, "Defending against fingerprint-copy attack in sensor-based camera identification," *IEEE Transactions on Information Forensics and Security*, vol. 6, no. 1, pp. 227–236, 2011.

[31] M. Begum and M. S. Uddin, "Digital image watermarking techniques: A review," *Information*, vol. 11, no. 2, p. 110, 2020.

[32] S. P. Vaidya, R. N. V. P. S. Kandala, P. V. S. S. R. Chandra Mouli, H. G. Zaini, A. Jaffar, P. Paramasivam, and S. S. M. Ghoneim, "A robust fragile watermarking approach for image tampering detection and restoration utilizing hybrid transforms," *Scientific Reports*, vol. 15, p. 17645, 2025.

[33] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, "Secure spread spectrum watermarking for multimedia," *IEEE Transactions on Image Processing*, vol. 6, no. 12, pp. 1673–1687, 1997.

[34] F. Hartung and B. Girod, "Watermarking of uncompressed and compressed video," *Signal Processing*, vol. 66, no. 3, pp. 283–301, 1998.

[35] M. Bistroń, J. M. Żurada, and Z. Piotrowski, "Deep learning for image watermarking: A comprehensive review and analysis of techniques, challenges, and applications," *Sensors*, vol. 26, no. 2, p. 444, 2026.

[36] S. Gowal *et al.*, "SynthID-Image: Image watermarking at internet scale," arXiv:2510.09263, 2025.

[37] P. Fernandez, H. Elsahar, I. Z. Yalniz, and A. Mourachko, "Video Seal: Open and efficient video watermarking," arXiv:2412.09492, 2024.

[38] T. Bui, S. Agarwal, and J. Collomosse, "TrustMark: Universal watermarking for arbitrary resolution images," arXiv:2311.18297, 2023.

[39] M. Kutter, S. Voloshynovskiy, and A. Herrigel, "The watermark copy attack," in *Proc. SPIE Security and Watermarking of Multimedia Contents II*, vol. 3971, 2000, pp. 371–380.

[40] P. Yang, H. Ci, Y. Song, and M. Z. Shou, "Steganalysis on digital watermarking: Is your defense truly impervious?" arXiv:2406.09026, 2024.

[41] X. Zhao, K. Zhang, Z. Su, S. Vasan, I. Grishchenko, C. Kruegel, G. Vigna, Y.-X. Wang, and L. Li, "Invisible image watermarks are provably removable using generative AI," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[42] M. Saberi, V. S. Sadasivan, K. Rezaei, A. Kumar, A. Chegini, W. Wang, and S. Feizi, "Robustness of AI-image detectors: Fundamental limits and practical attacks," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[43] P. Blythe and J. Fridrich, "Secure digital camera," in *Proc. Digital Forensic Research Workshop (DFRWS)*, 2004.

[44] M. Asikuzzaman and M. R. Pickering, "A survey on robust video watermarking algorithms for copyright protection," *Applied Sciences*, vol. 8, no. 10, p. 1891, 2018.

[45] Y. Tew and K. Wong, "HEVC watermarking techniques for authentication and copyright applications: Challenges and opportunities," *IEEE Access*, vol. 8, 2020.

[46] S. Zhu, K. Niu, J. Li, Y. Lu, and Q. Xu, "CSTFMark: Robust video watermarking against H.264/AVC compression via dynamic spatio-temporal frequency modulation and curriculum learning," *Journal of King Saud University – Computer and Information Sciences*, vol. 38, no. 3, 2026.

[47] A. Qureshi and D. Megías Jiménez, "Blockchain-based multimedia content protection: Review and open challenges," *Applied Sciences*, vol. 11, no. 1, p. 1, 2021.

[48] H. R. Hasan and K. Salah, "Combating deepfake videos using blockchain and smart contracts," *IEEE Access*, vol. 7, pp. 41 596–41 606, 2019.

[49] S. M. Darwish, M. M. Abu-Deif, and S. M. Elkaffas, "Blockchain for video watermarking: An enhanced copyright protection approach for video forensics based on perceptual hash function," *PLOS ONE*, vol. 19, no. 10, p. e0308451, 2024.

[50] S. L. Madapati and N. R. Pradhan, "Decentralizing video copyright protection: A novel blockchain-enabled framework with performance evaluation," *Frontiers in Artificial Intelligence*, vol. 8, 2025.

[51] National Institute of Standards and Technology, "Digital signature standard (DSS)," NIST, FIPS 186-5, 2023.

[52] J. Chen, N. Chen, Z. Wang, R. Dou, J. Liu, N. Wu, L. Liu, P. Feng, and G. Wang, "A review of recent advances in high-dynamic-range CMOS image sensors," *Chips (MDPI)*, vol. 4, no. 1, p. 8, 2025.

[53] W. Choi, H. Yoo, H. Kwon *et al.*, "Using 1:4 dual conversion gain technology for high SNR performance (fd-shared pixel)," in *Int. Conf. on Solid State Devices and Materials (SSDM)*, 2020.

[54] C. Park, H. Lee, E. Shim, J. Yun, K. Lee, Y. Jung, S. Yoon, I. Jeong, J. Ahn, and D. Chang, "World's first 16:4:1 triple conversion gain sensor with all-pixel AF for 82.4 db single-exposure HDR," *Electronic Imaging (IS&T), Image Sensors and Imaging Systems*, vol. 34, no. 7, pp. ISS–257, 2022.

[55] S. U. Sahu, "MOSFET-modulated dual conversion gain CMOS image sensors," Ph.D. dissertation, Boise State University, 2008.

[56] J. Zhang, J. P. Newman, Z. Wang, Y. Qian, P. Feliciano-Ramos, W. Guo, T. Honda, Z. S. Chen, C. Linghu, R. Etienne-Cummings, E. Fossum, and M. A. Wilson, "Pixel-wise programmability enables dynamic high-SNR cameras for high-speed microscopy," *Nature Communications*, vol. 15, p. 4480, 2024.

[57] Y. Luo, "CMOS image sensor design with programmable spatial-temporal exposure for machine vision and computational imaging," Ph.D. dissertation, University of British Columbia, 2020.

[58] J. Zhang, J. P. Newman, X. Wang, C. S. Thakur, J. Rattray, R. Etienne-Cummings, and M. A. Wilson, "A closed-loop all-electronic pixel-wise adaptive imaging system for high dynamic range video," *arXiv preprint arXiv:1906.10045*, 2019.

[59] A. Yang, E. Kang, W. Chen, H.-E. Lee, and A. C. Sankaranarayanan, "Towards spatially-varying gain and binning," *arXiv preprint arXiv:2507.04190*, 2025.

[60] B. Jiang, Y. Wu, Z. Zhong, K. Fan, Q. Lin, J. Liang, H. Liu, and X. Liu, "Cu–Cu hybrid bonding technology: From physical mechanisms to system integration for 3D ICs," *Moore and More (Springer)*, 2025.

[61] Y. Kagawa, N. Fujii, K. Aoyagi *et al.*, "An advanced Cu–Cu hybrid bonding for novel stacked CMOS image sensors," in *International Image Sensor Workshop (IISW)*, 2017.

[62] imec, "Wafer-to-wafer hybrid bonding: Pushing the boundaries to 400 nm interconnect pitch," 2024, https://www.imec-int.com.

[63] T.-H. Tsai, K.-H. Chang, A. Berkovich *et al.*, "A 400×400 3.24 μm 117 db-dynamic-range 3-layer stacked digital pixel sensor," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2025.

[64] M. Goto, Y. Honda, M. Nanba, Y. Iguchi, T. Saraya, M. Kobayashi, E. Higurashi, H. Toshiyoshi, and T. Hiramoto, "3-layer stacked pixel-parallel CMOS image sensors using hybrid bonding of SOI wafers," in *Electronic Imaging (IS&T), Imaging Sensors and Systems*, 2022, pp. ISS–258.

[65] M. R. I. Udoy, S. Alam, M. M. Islam, A. Jaiswal, and A. Aziz, "A review on digital pixel sensors," *arXiv preprint arXiv:2402.04507*, 2024.

[66] R. Nakamura, H. Tsugawa, W. Otsuka, K. Shimizu, Y. Kagawa, K. Ono, Y. Horie *et al.*, "A novel 1/1.3-inch 50 megapixel three-wafer-stacked CMOS image sensor with DNN circuit for edge processing," in *IEEE International Electron Devices Meeting (IEDM)*, 2024.

[67] S. J. Carey, A. Lopich, D. R. W. Barr, B. Wang, and P. Dudek, "A 100,000 fps vision sensor with embedded 535 GOPS/W 256×256 SIMD processor array," in *Symp. on VLSI Circuits*, 2013.

[68] B. An, M. Ding, T. Rabbani, A. Agrawal, Y. Xu, C. Deng, S. Zhu, A. Mohamed, Y. Wen, T. Goldstein, and F. Huang, "WAVES: Benchmarking the robustness of image watermarks," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[69] E. K. Company, "Kodak lossless true color image suite (photocd pcd0992)," https://r0k.us/graphics/kodak/, 24 reference photographs, accessed 2026.

[70] A. Novozamsky, B. Mahdian, and S. Saic, "IMD2020: A large-scale annotated dataset tailored for detecting manipulated images," in *IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV) Workshops*, 2020.

[71] J. Dong, W. Wang, and T. Tan, "CASIA image tampering detection evaluation database," in *IEEE China Summit and International Conf. on Signal and Information Processing (ChinaSIP)*, 2013.

[72] E. Brickell, J. Camenisch, and L. Chen, "Direct anonymous attestation," in *Proc. 11th ACM Conference on Computer and Communications Security (CCS)*, 2004, pp. 132–145.

[73] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," Internet Engineering Task Force, RFC 6962, 2013.